Jolly2DEngine

Developer Manual

Version 1.2.0

.NET 10 · Windows/WPF · DirectX 11 · x64

Setup, architecture, gameplay systems, tile maps and API reference

Compiled from the supplied documentation archive. The added tile-map chapter was cross-checked against the supplied engine source.

Contents

1. Documentation overview
2. Project setup and first game
3. Subsystem guide
4. Tile-map guide
5. Enumeration reference
6. Complete public API reference
7. API usage and parameter reference

> *Note: the API usage reference is mechanically generated and some parameter descriptions may be generic. When a signature or behaviour matters, verify it against the corresponding API declaration and engine source.*

Jolly2DEngine 1.2.0 Documentation

Jolly2DEngine is a .NET 10 Windows/WPF 2D game engine using DirectX 11 for rendering and NAudio for audio.

This documentation is intended to be the **developer manual and API reference**, not just a project description.

Start here

1. **Project setup and first game** **(docs/SETUP_AND_USING_THE_ENGINE.md)** — create a WPF project, add the engine, create `DirectXControl`, initialise the renderer, start `JollyGame`, create scenes and objects, and use the major systems.
2. **Subsystem guide** **(docs/SUBSYSTEM_GUIDE.md)** — understand lifecycle, threading, rendering, physics, input, audio, assets, saving, UI, particles and networking.
3. **Tile-map guide** **(docs/TILE_MAP_GUIDE.md)** — create tile grids, assign an atlas, render visible tiles, configure collision, animation, hazards and conveyors.
4. **Complete public API reference** **(docs/API_REFERENCE.md)** — every public declaration from the supplied source, grouped by source file.
5. **API usage and parameter reference** **(docs/API_USAGE_REFERENCE.md)** — every public method/constructor with a parameter-by-parameter explanation and return description.
6. **Enumeration reference** **(docs/ENUMS.md)** — every public enum and its values.

API documentation by subsystem

- `docs/api/Core_*.md` — game loop, game objects and game time
- `docs/api/Graphics_*.md` — sprites, textures and render snapshots
- `docs/api/Rendering_*.md` — camera, DirectX and sprite sheets
- `docs/api/Input_*.md` — keyboard, mouse, gamepad and input facade
- `docs/api/Physics_*.md` — colliders, collision tests, physics and character controllers
- `docs/api/Scene_*.md` — scenes and scene management
- `docs/api/UI_*.md` — UI elements and controls
- `docs/api/Audio_*.md` — audio manager
- `docs/api/Assets_*.md` — asset loading and lifetime
- `docs/api/Particles_*.md` — particle emitters and systems
- `docs/api/Save_*.md` — save-state and save-slot APIs
- `docs/api/Networking_*.md` — TCP client/server/message APIs
- `docs/api/Diagnostics_*.md` — profiling and diagnostics
- `docs/api/World_*.md` — tile maps

Each API page contains the declarations exactly as implemented by the engine source, while the setup and subsystem guides explain what the parameters are for and how the APIs fit together.

Engine architecture

```
                  WPF Application
                        |
                  DirectXControl
                        |
                  DirectXRenderer
                        |
               DirectXJollyRenderer
                        |
       +----------------+----------------+
       |                |                |
    Camera2D        JollyRenderer     DirectX 11
       |                |
       +---------- JollyRenderFrame
                        |
                JollySprite / Lines

JollyGame
   |
   +-- JollySceneManager
   |      +-- JollyScene
   |             +-- JollyGameObject
   |                    +-- JollySpriteObject
   |                    +-- Physics / Collider
   |
   +-- JollyInput
   +-- JollyGameTime
   +-- JollyDiagnostics
   +-- Audio / Assets / Save / Networking / UI / Particles
```

Important implementation facts

- Target: `net10.0-windows`.
- WPF and Windows Forms are enabled because the DirectX host uses WPF/WinForms interop.
- The project targets x64.
- DirectX shader bytecode is packaged under `Rendering/DirectX/Shaders`.
- The normal game update loop runs on a dedicated `Jolly2D Game Thread`.
- Input edge states should be read during update before `EndFrame()` advances them.
- `DirectXJollyRenderer` is the recommended high-level renderer; use `DirectXRenderer` directly only when building lower-level integrations.

API conventions

Unless a specific type says otherwise:

- Positions, sizes and velocities are expressed in engine world units.
- `float` values are used for world/render quantities.
- Colours use `JollyColor` with components in the `0..1` range; `JollyColor.FromBytes` is provided for `0..255` colour input.
- Audio volumes are clamped to `0..1`.
- Sprite opacity is clamped to `0..1`.
- Boolean input methods named `Is...Down` represent held state; `Is...Pressed` and `Is...Released` represent frame transitions.

Source of truth

The documentation was generated and written against the supplied Jolly2DEngine 1.2.0 source. If the API changes, regenerate/update the API inventory alongside the code.

Jolly2DEngine Project Setup Guide

1. Requirements

The supplied engine targets **.NET 10 for Windows**, WPF and x64. It uses DirectX 11 through Vortice and NAudio for audio.

Requirement	Version / setting
Target framework	`net10.0-windows`
Platform	x64 recommended / required for the supplied renderer build
UI	WPF + Windows Forms enabled
DirectX	Direct3D 11 / DXGI through Vortice 3.8.1
Audio	NAudio 3.1.0

The engine project itself already declares:

```
<TargetFramework>net10.0-windows</TargetFramework>
<UseWPF>true</UseWPF>
<UseWindowsForms>true</UseWindowsForms>
<PlatformTarget>x64</PlatformTarget>
```

If you consume the NuGet package, prefer an x64 WPF application so the native graphics stack and the engine's x64 assumptions match.

2. Create the game project

Create a WPF application targeting .NET 10 for Windows, then reference `Jolly2DEngine` either as a project reference or NuGet package.

For a local project reference:

```
<ItemGroup>
  <ProjectReference Include="..\Jolly2DEngine\Jolly2DEngine.csproj" />
</ItemGroup>
```

For a package reference:

```
<ItemGroup>
  <PackageReference Include="Jolly2DEngine" Version="1.2.0" />
</ItemGroup>
```

The engine package carries its DirectX shader bytecode as content. Do not remove the shader content from the published package.

3. Add the DirectX control to the main window

`DirectXControl` is the WPF host for the engine's DirectX renderer.

`MainWindow.xaml`:

```
<Window x:Class="MyGame.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:dx="clr-namespace:Jolly2DEngine.Rendering.DirectX;assembly=Jolly2DEngine"
        Title="My Jolly2D Game"
        Width="1280"
        Height="720">
    <Grid>
        <dx:DirectXControl x:Name="DirectX" />
    </Grid>
</Window>
```

The control creates/owns the low-level DirectX renderer. Wait for it to be ready before creating resources that require the DirectX device.

```
private async void Window_Loaded(object sender, RoutedEventArgs e)
{
    await DirectX.WaitForReadyAsync();
    // Safe place to create DirectX-backed resources.
}
```

4. Create the Jolly renderer

```
DirectXJollyRenderer renderer = DirectX.CreateJollyRenderer();
```

The renderer owns a `Camera2D` and exposes the higher-level Jolly rendering API.

```
renderer.Camera.SetViewport((float)DirectX.ActualWidth, (float)DirectX.ActualHeight);
renderer.ClearColor = Colors.Black;
renderer.LineGlowEnabled = true;
renderer.LineGlowWidth = 2.0f;
renderer.LineGlowStrength = 1.0f;
renderer.LineGlowFalloff = 2.0f;
```

5. Create the game loop

Derive from `JollyGame` and override `OnInitialize`, `OnUpdate`, and optionally `OnShutdown`.

```
public sealed class MyGame : JollyGame
{
    private readonly DirectXJollyRenderer _renderer;

    public MyGame(DirectXJollyRenderer renderer)
    {
        _renderer = renderer;
        GameLoopFPS = 240.0;
        RenderFPS = 120.0;
    }

    protected override void OnInitialize()
    {
        // Create scenes and game objects here.
    }

    protected override void OnUpdate(JollyGameTime gameTime)
    {
        // Game logic runs here.
    }

    protected override void OnShutdown()
    {
        // Release game-specific resources here.
    }
}
```

Start and stop it from the WPF lifetime:

```
private MyGame? _game;

private void Window_Loaded(object sender, RoutedEventArgs e)
{
    _game = new MyGame(DirectX.CreateJollyRenderer());
    _game.Start();
}

private void Window_Closed(object? sender, EventArgs e)
{
    _game?.Stop();
    _game?.Dispose();
}
```

Important threading rule

`JollyGame.Start()` creates a dedicated **Jolly2D Game Thread**. `OnUpdate` and scene updates therefore run on that thread, not the WPF UI thread. Avoid directly modifying WPF controls from `OnUpdate`.

The DirectX renderer has its own render-side processing. Treat WPF objects as UI-thread owned and use engine state/events for communication between game and UI.

6. Create a scene

A scene derives from `JollyScene`.

```
public sealed class GameScene : JollyScene
{
    public override void OnEnter()
    {
        // Scene became active.
    }

    public override void Initialize()
    {
        base.Initialize();
```

```
        // Create or initialise scene objects.
    }

    public override void OnExit()
    {
        // Scene is leaving the active state.
    }
}
```

Add it through `JollySceneManager`:

```
GameScene scene = new();
scene.Name = "Game";
Scenes.Add(scene);
Scenes.SetActive(scene);
```

The exact scene-manager operations available in your source are listed in `API_REFERENCE.md` under `Scene/JollySceneManager.cs`.

7. Create textures and sprites

Use `DirectXJollyRenderer.LoadTexture` for file-backed textures:

```
JollyTexture texture = _renderer.LoadTexture("Assets/Player.png");
JollySprite sprite = _renderer.CreateSprite(texture);
sprite.Position = new JollyVector2(100.0f, 100.0f);
sprite.Scale = new JollyVector2(2.0f, 2.0f);
sprite.Color = JollyColor.White;
```

For generated graphics:

```
JollyTexture white = _renderer.CreateSolidTexture(255, 255, 255);
JollyTexture circle = _renderer.CreateCircleTexture(0, 255, 255, 255, 32);
```

8. Use `JollySpriteObject`

`JollySpriteObject` wraps a `JollySprite` in the scene/game-object lifecycle and keeps the sprite synchronized with `Position`, `Scale`, `Rotation`, and `Visible`.

```
JollySpriteObject player = new(_renderer.CreateSprite(texture));
player.Name = "Player";
player.Position = new JollyVector2(200.0f, 500.0f);
player.SetRectangleCollider(player.Sprite.Width, player.Sprite.Height);
scene.Add(player);
```

9. Input

Attach the engine input system to the WPF window:

```
_game.Input.Attach(this);
```

Keyboard:

```
if (_game.Input.IsKeyDown(Key.A))
{
    player.Position = new JollyVector2(player.Position.X - 200.0f * (float)_game.GameTime.DeltaSeconds, player.
Position.Y);
}

if (_game.Input.IsKeyPressed(Key.Space))
{
    // One-shot press.
}
```

Mouse:

```
System.Windows.Point mouse = _game.Input.MousePosition;
Vector2 world = _game.Input.GetMouseWorldPosition(_renderer.Camera);
```

Gamepads are exposed through `Input.Gamepads`; button and analogue-stick helpers are provided by `JollyGamepad`.

Detach when the window is no longer using the input source:

```
_game.Input.Detach(this);
```

10. Camera

The renderer exposes `renderer.Camera`.

```
renderer.Camera.Position = new Vector2(400.0f, 300.0f);
```

```
renderer.Camera.Zoom = 1.0f;
renderer.Camera.Rotation = 0.0f;
renderer.Camera.Follow(player.Position);
```

For smooth following:

```
renderer.Camera.FollowSmooth(player.Position, (float)gameTime.DeltaSeconds);
```

World/screen conversion:

```
Vector2 screen = renderer.Camera.WorldToScreen(worldPosition);
Vector2 world = renderer.Camera.ScreenToWorld(screenPosition);
```

11. Physics

Enable physics on a game object and give it a collider:

```
player.UsePhysics = true;
player.UseGravity = true;
player.Gravity = new JollyVector2(0.0f, 1200.0f);
player.SetRectangleCollider(32.0f, 48.0f);
```

Circle collider:

```
player.SetCircleCollider(16.0f);
```

Trigger:

```
player.SetTrigger(true);
```

Collision layers:

```
player.SetCollisionLayers(
    JollyCollisionLayer.Layer0,
    JollyCollisionLayer.Layer1 | JollyCollisionLayer.Layer2);
```

Collision callbacks are overridden on the game object:

```
public override void OnCollisionEnter(JollyGameObject other)
{
    // Physical collision started.
}

public override void OnTriggerEnter(JollyGameObject other)
{
    // Trigger overlap started.
}
```

12. Character controller

```
JollyCharacterController controller = player.AddCharacterController();
controller.MaxMoveSpeed = 300.0f;
controller.JumpVelocity = 450.0f;

controller.Move(horizontalInput);
controller.SetJumpHeld(jumpHeld);

if (jumpPressed)
{
    controller.RequestJump();
}
```

The controller provides coyote time and jump buffering. `JollyGameObject.IsGrounded` is updated by collision resolution.

13. Animations

Create a `JollySpriteSheet`, then a `JollyAnimation` describing the frame range and playback rate. Add it to the sprite and play it using the sprite's animation API.

```
JollyAnimation animation = new(
    "Run",
    sheet,
    startFrame: 0,
    frameCount: 8,
    fps: 12.0,
    loop: true);
```

Subscribe to completion when using non-looping animations:

```
sprite.AnimationCompleted += (sender, name) =>
{
    // Animation named 'name' finished.
```

```
};
```

See `Animation/JollyAnimation.cs` and `Graphics/JollySprite.cs` in the API reference for the complete animation API.

14. Audio

Create a `JollyAudioManager`, load effects, then play them by name:

```
JollyAudioManager audio = new();
audio.LoadSound("Shoot", "Assets/Audio/Shoot.wav");
audio.PlaySound("Shoot");
```

Music is streamed from its path:

```
audio.PlayMusic("Assets/Audio/Music.wav", loop: true);
audio.PauseMusic();
audio.ResumeMusic();
audio.StopMusic();
```

Volume is normalised to `0..1`:

```
audio.MasterVolume = 1.0f;
audio.MusicVolume = 0.75f;
audio.SfxVolume = 1.0f;
audio.Muted = false;
```

Dispose the manager during shutdown.

15. Asset manager

For larger projects use `JollyAssetManager` to standardise paths and reference-count textures.

```
JollyAssetManager assets = new(renderer, audio, "Assets");
JollyTexture playerTexture = assets.LoadTexture("Player.png");
```

The default asset folders are:

- `Textures`
- `Audio`
- `Fonts`
- `Maps`
- `Animations`

Use `AcquireTexture` when you want an explicit reference-counted handle and dispose the returned `JollyAssetHandle<T>` when finished.

16. Tile maps

`JollyTileMap` stores tile occupancy/data and `JollyTileMapRenderer` renders it. A scene can associate a tile map with:

```
scene.SetTileMap(tileMap, worldX, worldY);
```

The physics system uses solid tiles during collision resolution for physics-enabled objects.

17. Particles

Use `JollyParticleSystem` and `JollyParticleEmitter` for effects. The built-in presets include:

- `Sparks`
- `Smoke`
- `Fire`
- `Explosion`

Emitter shapes include point, circle, rectangle and line. Configure lifetime, speed, size, direction, spread, gravity, drag, rotation speed, colour and opacity through the emitter/preset properties.

18. UI

The UI system is separate from world sprites. `JollyUIManager` owns `JollyUIElement` instances and handles hover/press state using screen-space mouse coordinates.

Available controls include:

- `JollyUIPanel`
- `JollyUIText`
- `JollyUIButton`
- `JollyUISlider`
- `JollyUIProgressBar`
- `JollyUI9Slice`

Use the element layout/anchor APIs rather than manually converting every control position from world coordinates.

19. Saving

Capture a scene:

```
JollySceneSaveState state = JollySceneSaveState.Capture(scene);
```

Persist arbitrary game state with `JollySaveManager`:

```
JollySaveManager saves = new();
saves.SaveGame("slot1", new MySaveData { Level = 3, Score = 12500 }, gameVersion: 1);

MySaveData loaded = saves.LoadGame<MySaveData>("slot1", expectedGameVersion: 1);
```

Use `TryLoadGame` when a missing or invalid save is expected and should not throw.

20. Diagnostics and profiling

`JollyGame` creates `Diagnostics` automatically.

```
JollyProfile? profile = game.Diagnostics.RegisterProfile("AI");

using (game.Diagnostics.MeasureUpdate())
{
    using (new JollyProfileScope(profile))
    {
        UpdateAI();
    }
}
```

The diagnostics system tracks update/render timing and entity counts and exposes snapshots/formatters for displaying the information.

21. Line rendering

For Beam Rider, Tron-style trails and other vector effects, submit `JollyLineSnapshot` instances as part of a `JollyRenderFrame`.

The renderer supports configurable line glow:

```
renderer.LineGlowEnabled = true;
renderer.LineGlowWidth = 2.0f;
renderer.LineGlowStrength = 1.0f;
renderer.LineGlowFalloff = 2.0f;
```

`JollyLineSnapshot` supports separate start/end thickness values, allowing tapered projectiles and trails.

22. Networking

`JollyNetworkServer` listens for clients and raises connection/message events. `JollyNetworkClient` connects to a server. `JollyNetworkMessage` supports raw bytes or UTF-8 text.

Server:

```
JollyNetworkServer server = new();
server.ClientConnected += connection => { };
```

```
server.MessageReceived += (connection, message) => { };
server.Start(7777);
server.BroadcastText("Welcome");
```

Client:

```
JollyNetworkClient client = new();
client.MessageReceived += message => { };
await client.ConnectAsync("127.0.0.1", 7777);
client.SendText("Hello");
```

Always stop/dispose networking objects during application shutdown.

23. Low-level DirectX API

Most games should use `DirectXJollyRenderer` rather than `DirectXRenderer` directly. The low-level renderer exists for engine-level integrations and exposes:

- device/window initialisation
- render-loop control
- image/texture creation
- frame submission
- resize
- image filtering
- stretch modes
- VSync/FPS control
- line glow
- render statistics

Use `DirectXControl.CreateJollyRenderer()` for the normal application path.

24. Typical project structure

```
MyGame/
  Assets/
    Textures/
    Audio/
    Maps/
    Animations/
  Scenes/
    MainMenuScene.cs
    GameScene.cs
  Objects/
    Player.cs
    Enemy.cs
    Projectile.cs
  UI/
    Hud.cs
  MainWindow.xaml
  MainWindow.xaml.cs
  MyGame.cs
```

Keep engine-independent gameplay code in scene/game-object classes. Keep WPF code in the window/UI layer and keep DirectX resource creation behind the renderer/asset layer.

Jolly2DEngine Subsystem Guide

Core lifecycle

`JollyGame` is the application-level owner. It creates `JollySceneManager`, `JollyInput`, `JollyGameTime`, and `JollyDiagnostics`.

The lifecycle is:

1. `Start()` creates the game thread.
2. `OnInitialize()` is called once on that thread.
3. Each frame updates `JollyGameTime`.
4. `Scenes.Update(gameTime)` runs.
5. `OnUpdate(gameTime)` runs.
6. Input's frame state is advanced with `EndFrame()`.
7. On stop, `OnShutdown()` runs and scenes are shut down.

`GameLoopFPS = 0` means the game loop is not deliberately capped. A positive value supplies a target update rate. `RenderFPS` is forwarded to the renderer through `OnRenderFPSChanged`.

Game objects

`JollyGameObject` is the base world entity. Its transform is `Position`, `Scale`, and `Rotation`. `Velocity`, `Acceleration`, `Gravity`, `Drag`, and the physics flags control simulation. `Collider` is optional.

Override:

- `Initialize()` for one-time object setup.
- `Update(JollyGameTime)` for object logic.
- `Draw()` for custom draw integration.
- `Shutdown()` for cleanup.
- collision/trigger callbacks for interaction.

Call `Destroy()` to mark the object destroyed and inactive. The scene removes destroyed objects as part of its update lifecycle.

`JollySpriteObject` adds a `JollySprite` and synchronises the sprite transform automatically. Its collision helpers are the recommended way to configure sprite-based collision.

Scene lifecycle

`JollyScene` owns an object list and supports deferred add/remove while an update is in progress. This prevents collection modification problems during object callbacks.

`OnEnter()` and `OnExit()` are scene transitions. `Initialize()` initialises objects once. `SetTileMap()` associates a tile map and world offset with the scene.

Rendering model

The normal path is:

```
WPF Window
   -> DirectXControl
      -> DirectXRenderer
   -> DirectXJollyRenderer
      -> JollyTexture / JollySprite
      -> JollyRenderFrame
      -> Camera2D
```

A `JollySprite` is a high-level render object. At frame submission it becomes a `JollySpriteSnapshot`; line primitives become `JollyLineSnapshot` values. The renderer consumes the resulting `JollyRenderFrame`.

The renderer batches sprites and exposes counters such as `RenderSpriteCount`, `RenderBatchCount`, and `RenderDrawCallCount` for profiling.

Coordinate system

World and screen coordinates use `float` values. `Camera2D` transforms world positions into screen coordinates and supports zoom, rotation, viewport sizing, bounds and screen shake.

`Follow()` immediately targets a position. `FollowSmooth()` applies a delta-time-aware interpolation. `Shake(intensity, duration)` creates a temporary offset; call `UpdateShake(deltaSeconds)` when driving a camera manually.

Physics

Collision filtering uses bitmasks. An object collides only when both masks accept the opposing layer:

```
(moving.Mask & other.Layer) != 0
and
(other.Mask & moving.Layer) != 0
```

Collider shapes are rectangle, circle and pixel. Trigger colliders participate in overlap callbacks but are not resolved as solid physical collisions.

`JollyPhysicsSystem` performs the engine's movement/collision resolution. `JollyCollisionSystem.Check` provides direct collision tests for colliders.

Input state semantics

`IsKeyDown`/`IsButtonDown` are level tests: true while held.

`IsKeyPressed`/`IsButtonPressed` are edge tests for the current frame.

`IsKeyReleased`/`IsButtonReleased` are release-edge tests.

The game calls `Input.EndFrame()` after update processing, so perform input queries during the update phase.

Audio

Sound effects are cached in memory after `LoadSound`. `PlaySound` creates a playback instance from the cached samples, allowing overlapping effects. Music is handled separately through an `AudioFileReader` and can loop.

Master volume multiplies both music and SFX volume. `Muted` forces effective output volume to zero without losing the configured volume values.

Assets

`JollyAssetManager` centralises asset paths and texture lifetime. Texture handles can be reference-counted using `AcquireTexture`/`JollyAssetHandle<T>.Dispose()`.

`GetTexturePath`, `GetAudioPath`, `GetMapPath`, `GetAnimationPath` and `GetFontPath` resolve names against the configured asset root.

Save system

`JollySaveManager` wraps JSON save files with a format version and game version. A backup is maintained by the manager and can be restored through `RestoreBackup`.

`JollySceneSaveState` captures engine-owned object state. It does not magically serialize arbitrary fields added by a custom game object; store custom game data in your own payload type.

UI

UI elements are screen-space objects. `JollyUIManager` tracks the hovered and pressed element and feeds mouse interaction into controls. Buttons expose `Click`; sliders expose `ValueChanged`.

Nine-slice elements are intended for scalable panels/buttons where the corners should retain their original dimensions.

Particles

`JollyParticleEmitter` creates transient particles according to an emitter configuration. Presets are convenient starting configurations, not special renderer modes. Adjust the preset properties after construction when you need a custom effect.

Networking

Networking is TCP-based. `JollyNetworkMessage.Payload` is the raw message body; `Text` decodes it as UTF-8. TCP provides ordered reliable delivery but does not define a game protocol, message schema, authentication or prediction model. Build those above the supplied transport layer.

Threading and disposal

Dispose long-lived engine resources in reverse ownership order: stop the game, stop/dispose networking, stop audio, then release renderer/assets as appropriate. Never assume a WPF control can be touched from the game thread.

Tile-map Guide

This guide documents the tile-map functionality present in Jolly2DEngine 1.2.0. It is based on `World/JollyTileMap.cs`, `World/JollyTileMapRenderer.cs`, `Scene/JollyScene.cs`, and `Physics/JollyPhysicsSystem.cs` in the supplied engine source.

What the current tile-map system provides

- A fixed-size grid with configurable tile width and height.
- A single texture atlas divided into equal-sized tile cells.
- `-1` as the empty-cell value; non-negative values are atlas tile indices.
- Tile-to-world and world-to-tile coordinate helpers.
- A cached sprite renderer that updates and submits only tiles in the camera's visible world bounds.
- Per-cell solid flags.
- Per-tile-index animation, hazard and conveyor settings.
- Scene integration through `JollyScene.SetTileMap` and physics resolution for game objects using physics and a non-trigger collider.

Coordinate and atlas rules

Tile coordinates start at `(0, 0)` in the top-left of the map. `GetTilePosition(x, y)` returns the tile's local world position as `(x TileWidth, y TileHeight)`. The map's scene position is separate: pass it to `JollyScene.SetTileMap(map, x, y)` and set the same `X` and `Y` on the map renderer.

The atlas image must be divisible evenly by the tile width and height. For example, a 256 × 128 atlas with 32 × 32 tiles has 8 columns, 4 rows and 32 tile indices. Indices are row-major, starting at zero in the top-left. `SetTexture` validates divisibility; it does not resize the source image or infer a different tile size.

Create and populate a map

```
using Jolly2DEngine.World;
using Jolly2DEngine.Graphics;
using Jolly2DEngine.Rendering.DirectX;

// Load a tileset atlas whose cell size is 32 × 32 pixels.
JollyTexture atlas = renderer.LoadTexture("Assets/Tiles.png");

// Create a 40-column by 20-row map; each tile occupies 32 × 32 world units.
JollyTileMap map = new(40, 20, 32.0f, 32.0f);
map.SetTexture(atlas);

// Place tiles by atlas index. -1 means the cell is empty.
map.SetTile(0, 19, 0);
map.SetTile(1, 19, 1);
map.SetTile(2, 19, 1);
map.SetTile(3, 19, 2);

// Solid state is per map cell, not automatically inferred from the atlas index.
map.SetSolid(0, 19, true);
map.SetSolid(1, 19, true);
map.SetSolid(2, 19, true);
map.SetSolid(3, 19, true);
```

The code above uses `renderer` as an already-created `DirectXJollyRenderer`. Ensure the texture is created by that renderer, because the renderer only accepts its own DirectX textures.

Create the tile-map renderer

```
JollyTileMapRenderer mapRenderer = new(map, renderer.CreateSprite)
{
    X = 0.0f,
    Y = 0.0f,
    Layer = 0
};

mapRenderer.Rebuild();
```

Call `Update` with the camera and elapsed time before assembling the render frame. Then include the returned `Sprites` alongside the game's other sprites when creating a `JollyRenderFrame`.

```
mapRenderer.Update(renderer.Camera, (float)gameTime.DeltaSeconds);

List<JollySprite> visibleSprites = new();
visibleSprites.AddRange(mapRenderer.Sprites);
visibleSprites.AddRange(otherSprites);

renderer.SubmitFrame(new JollyRenderFrame(visibleSprites));
```

This is an integration pattern, not a complete `JollyGame` subclass. Your game's render scheduling must call the update/submission code at the appropriate point. The tile-map renderer caches sprite instances by map coordinate and calculates a visible tile range from the camera. When you change map cells at runtime, `Update` reflects those tile values; call `Rebuild` when you want to discard and recreate the sprite cache.

Collision and scene integration

The scene can store one tile map and its world-space origin:

```
scene.SetTileMap(map, 0.0f, 0.0f);
```

`JollyPhysicsSystem.ResolveTileMap` / `JollyScene.ResolveTileMap` resolve tile collisions for game objects that have physics enabled and an enabled, non-trigger collider. Solid cells block movement and can set `IsGrounded` on landing. A cell's solid flag is separate from the visual tile index, so set both explicitly when needed.

For a moving map, use the same origin coordinates consistently for rendering and collision. `JollyTileMapRenderer.X/Y` affect visual placement; `JollyScene.SetTileMap`'s `x/y` are used for collision placement. They are separate properties and are not automatically synchronised.

Animated, hazard and conveyor tiles

These settings are associated with the **atlas tile index**, rather than a single grid cell. Every occurrence of that tile index shares the setting.

```
// Animate atlas tiles starting at index 10, cycling through 4 frames at 8 FPS.
map.SetAnimatedTile(tileIndex: 10, startFrame: 10, frameCount: 4, framesPerSecond: 8.0f, loop: true);

// Mark every occurrence of atlas tile 20 as a hazard.
map.SetHazardTile(20, true);
map.HazardContact += (gameObject, tileX, tileY, tileIndex) =>
{
    // Apply game-specific damage or other hazard behaviour here.
};

// A conveyor tile pushes grounded physics objects horizontally at this speed.
map.SetConveyorTile(21, 80.0f);
```

The physics system raises `HazardContact` when a qualifying physics collider intersects a hazard tile. The engine reports the contact; your game decides what damage or response to apply. Conveyor speed is applied when the collision normal indicates the object is standing on the tile. A zero conveyor speed removes the conveyor setting.

For a non-looping animation, the selected frame stops at the final frame. `RemoveAnimatedTile`, `RemoveConveyorTile` and `SetHazardTile(tileIndex, false)` remove the corresponding behaviour.

Query map data

```
int tileIndex = map.GetTile(tileX, tileY);
bool solid = map.IsSolid(tileX, tileY);
System.Numerics.Vector2 tileOrigin = map.GetTilePosition(tileX, tileY);
System.Numerics.Vector2 tileCoordinate = map.WorldToTile(worldX, worldY);

if (map.TryGetTileAtWorldPosition(worldX, worldY, out int foundTileIndex))
{
    // A non-empty tile was found at this world coordinate.
}
```

`GetTile` returns `-1` for coordinates outside the map. `IsSolid` returns `false` outside the map. `TryGetTileAtWorldPosition` returns `false` for positions outside the map or for an empty cell. `WorldToTile` returns a `Vector2` containing floored tile coordinates; convert those components to integers when using them as grid indices.

Current limitations to be aware of

The supplied implementation is a useful runtime tile-map foundation, but it is not a full map-authoring pipeline:

- There is no built-in Tiled (`.tmx` / `.tsx`) importer in the supplied tile-map classes.
- `JollyTileMap` represents one grid and one atlas; there is no first-class collection of named tile layers in these classes.
- The solid state is per cell, while animation, hazard and conveyor configuration is shared by atlas tile index.
- There is no built-in visual tile-map editor or map-file serializer in `JollyTileMap`.
- Tile animation is updated using the renderer's elapsed animation time; animation settings are based on atlas indices.

These are possible future improvements, not features this version already promises. For an initial game, create maps in code or write a small JSON loader that fills the grid, tile indices and solid flags.

API reference

Member	Purpose
`JollyTileMap(width, height, tileWidth, tileHeight)`	Allocate a fixed grid.
`SetTexture(texture)`	Assign an evenly divisible tile atlas.
`SetTile(x, y, tileIndex)` / `GetTile(x, y)`	Write/read a cell; `-1` is empty.
`SetSolid(x, y, solid)` / `IsSolid(x, y)`	Set/read a cell's solid flag.
`GetTilePosition(x, y)`	Return the local world origin of a cell.
`WorldToTile(x, y)`	Convert world coordinates to floored grid coordinates.
`TryGetTileAtWorldPosition(x, y, out tileIndex)`	Query a non-empty tile at a world position.
`SetAnimatedTile(...)`	Associate animation frames and rate with an atlas index.
`SetHazardTile(tileIndex, hazard)`	Mark/remove an atlas index as hazardous.
`SetConveyorTile(tileIndex, speed)`	Associate conveyor speed with an atlas index.
`JollyTileMapRenderer(map, spriteFactory)`	Create the sprite cache renderer.
`Rebuild()` / `Update(camera, deltaSeconds)`	Recreate the cache / update visible sprites and animation.

See `docs/api/World_JollyTileMap.md`, `docs/api/World_JollyTileMapRenderer.md`, and the physics/scene API pages for declaration details.

Enumerations

JollyGamepadButton

- A
- B
- X
- Y
- Back
- Start
- LeftShoulder
- RightShoulder
- LeftStick
- RightStick
- DPadUp
- DPadDown
- DPadLeft
- DPadRight

JollyMouseButton

- Left
- Middle
- Right
- X1
- X2

JollyParticleEmitterShape

- Point
- Circle
- Rectangle
- Line

JollyParticlePreset

- Sparks
- Smoke
- Fire
- Explosion

JollyColliderShape

- None
- Rectangle
- Circle
- Pixel

`JollyCollisionLayer`

- None = 0
- Layer0 = 1u << 0
- Layer1 = 1u << 1
- Layer2 = 1u << 2
- Layer3 = 1u << 3
- Layer4 = 1u << 4
- Layer5 = 1u << 5
- Layer6 = 1u << 6
- Layer7 = 1u << 7
- Layer8 = 1u << 8
- Layer9 = 1u << 9
- Layer10 = 1u << 10
- Layer11 = 1u << 11
- Layer12 = 1u << 12
- Layer13 = 1u << 13
- Layer14 = 1u << 14
- Layer15 = 1u << 15
- Layer16 = 1u << 16
- Layer17 = 1u << 17
- Layer18 = 1u << 18
- Layer19 = 1u << 19
- Layer20 = 1u << 20
- Layer21 = 1u << 21
- Layer22 = 1u << 22
- Layer23 = 1u << 23
- Layer24 = 1u << 24
- Layer25 = 1u << 25
- Layer26 = 1u << 26
- Layer27 = 1u << 27
- Layer28 = 1u << 28
- Layer29 = 1u << 29
- Layer30 = 1u << 30
- Layer31 = 1u << 31
- All = uint.MaxValue

`JollyCollisionMode`

- None
- Bounds
- Pixel

DirectXImageFilter

- Point
- Linear

DirectXStretchMode

- None
- Stretch
- Uniform
- UniformToFill

JollySpriteSheetOrder

- Horizontal
- Vertical

JollyUIAnchor

- TopLeft
- Top
- TopRight
- Left
- Center
- Right
- BottomLeft
- Bottom
- BottomRight
- Stretch

JollyUIAnimationProperty

- Position
- Size
- Opacity

JollyUILayout

- None
- Vertical
- Horizontal

JollyUIProgressBarOrientation

- Horizontal
- Vertical

JollyUISliderOrientation

- Horizontal
- Vertical

Jolly2DEngine API Reference

This is the public API inventory for the engine source supplied with this documentation. Every public declaration is listed with its source file. Use the subsystem guides for explanations and examples.

How to read this reference

- **Properties/fields** are values you configure or read.
- **Methods** perform an engine operation; parameters are shown exactly as declared by the source.
- **Events** are callbacks raised by the engine.
- **Constructors** show the values required to create the type.
- `CancellationToken cancellationToken = default` means cancellation is optional.

Animation/JollyAnimation.cs

```
public sealed class JollyAnimation

public string Name { get; }  public JollySpriteSheet Sheet { get; }  public int StartFrame { get; }  public int FrameCount { get; }  public double FPS { get; }  public bool Loop { get; }  public JollyAnimation(string name, JollySpriteSheet sheet, int startFrame, int frameCount, double fps, bool loop = true)

public JollySpriteSheetFrame GetFrame(int animationFrame)
```

Assets/JollyAssetHandle.cs

```
public sealed class JollyAssetHandle<T> : IDisposable where T : class

public T Asset { get; }  public bool IsReleased => _release == null;

public void Dispose()
```

Assets/JollyAssetManager.cs

```
public sealed class JollyAssetManager : IDisposable

public const string TexturesFolder = "Textures";

public const string AudioFolder = "Audio";

public const string FontsFolder = "Fonts";

public const string MapsFolder = "Maps";

public const string AnimationsFolder = "Animations";

public JollyAssetManager(DirectXJollyRenderer renderer, JollyAudioManager? audio = null, string? rootPath = null)

public string RootPath { get; }  public string TexturesPath => Path.Combine(RootPath, TexturesFolder);

public string AudioPath => Path.Combine(RootPath, AudioFolder);

public string FontsPath => Path.Combine(RootPath, FontsFolder);

public string MapsPath => Path.Combine(RootPath, MapsFolder);

public string AnimationsPath => Path.Combine(RootPath, AnimationsFolder);

public JollyTexture LoadTexture(string assetName)

public async Task<JollyTexture> LoadTextureAsync(string assetName, CancellationToken cancellationToken = default)

public JollyAssetHandle<JollyTexture> AcquireTexture(string assetName)

public void ReleaseTexture(string assetName)

public bool IsTextureLoaded(string assetName)

public void LoadSound(string assetName)

public void PlaySound(string assetName)

public void PlayMusic(string assetName, bool loop = true)

public void UnloadSound(string assetName)

public string GetTexturePath(string assetName) => ResolveAssetPath(TexturesFolder, NormalizeAssetName(assetName));
```

```
public string GetAudioPath(string assetName) => ResolveAssetPath(AudioFolder, NormalizeAssetName(assetName));

public string GetFontPath(string assetName) => ResolveAssetPath(FontsFolder, NormalizeAssetName(assetName));

public string GetMapPath(string assetName) => ResolveAssetPath(MapsFolder, NormalizeAssetName(assetName));

public string GetAnimationPath(string assetName) => ResolveAssetPath(AnimationsFolder, NormalizeAssetName(
assetName));

public bool AssetExists(string category, string assetName)

public void Validate(params JollyAssetRequirement[] requirements)

public void Dispose()

public TextureEntry(JollyTexture texture)

public JollyTexture Texture { get; } public int ReferenceCount { get; set; } } }  public sealed record
JollyAssetRequirement(string Category, string Name);
```

Audio/JollyAudioManager.cs

```
public sealed class JollyAudioManager : IDisposable

public float MasterVolume

public float MusicVolume

public float SfxVolume

public bool Muted

public bool IsMusicPlaying => _musicOutput?.PlaybackState == PlaybackState.Playing;

public string? CurrentMusic => _musicPath;

public void LoadSound(string name, string path)

public void UnloadSound(string name)

public void PlaySound(string name)

public void StopAllSounds()

public void PlayMusic(string path, bool loop = true)

public void PauseMusic()

public void ResumeMusic()

public void StopMusic()

public void ClearSounds()

public void Dispose()

public CachedSound Sound { get; }  public WaveOut Output { get; }  public SoundInstance(CachedSound sound,
float volume)

public void Dispose()

public WaveFormat WaveFormat { get; }  public float[] AudioData { get; }  public CachedSound(string
audioFileName)

public WaveFormat WaveFormat => _sound.WaveFormat;

public CachedSoundSampleProvider(CachedSound sound)

public int Read(Span<float> buffer)

public LoopStream(WaveStream source)

public override WaveFormat WaveFormat => _source.WaveFormat;

public override long Length => _source.Length;

public override long Position

public override int Read(byte[] buffer, int offset, int count)
```

Core/JollyGame.cs

```
public abstract class JollyGame : IDisposable

public JollySceneManager Scenes { get; }  public JollyInput Input { get; }  public JollyGameTime GameTime { get;
 } = new JollyGameTime();

public JollyDiagnostics Diagnostics { get; }  public double GameLoopFPS

public double GameLoopActualFPS => Volatile.Read(ref _gameLoopActualFPS);
```

```
public double GameFrameTime => GameTime.FrameTime;

public double RenderFPS

public bool IsRunning => _running;

public void Start()

public void Stop()
```

Core/JollyGameObject.cs

```
public abstract class JollyGameObject

public string SaveId { get; set; } = Guid.NewGuid().ToString("N");

public string Name { get; set; } = string.Empty;

public string Tag { get; set; } = string.Empty;

public bool Active { get; set; } = true;

public JollyVector2 Position { get; set; } public JollyVector2 Scale { get; set; } = JollyVector2.One;

public float Rotation { get; set; } public bool Visible { get; set; } = true;

public JollyVector2 Velocity { get; set; } public JollyVector2 Acceleration { get; set; } public JollyVector2
Gravity { get; set; } public float Drag { get; set; } public bool UsePhysics { get; set; } public bool
UseGravity { get; set; } public bool IsGrounded { get; set; } public bool WasGrounded { get; set; } public
float GroundFriction { get; set; } = 12.0f;

public float AirFriction { get; set; } = 0.0f;

public float Restitution { get; set; } = 0.0f;

public JollyCollider? Collider { get; private set; } private JollyCharacterController? _characterController;

public JollyCharacterController? CharacterController => _characterController;

public bool IsDestroyed { get; private set; } internal JollyScene? Scene { get; set; }  public virtual void
Initialize()

public virtual void Update(JollyGameTime gameTime)

public virtual void Draw()

public virtual void Shutdown()

public void SetCollider(JollyCollider collider)

public void RemoveCollider()

public virtual void OnCollisionEnter(JollyGameObject other)

public virtual void OnCollisionStay(JollyGameObject other)

public virtual void OnCollisionExit(JollyGameObject other)

public virtual void OnTriggerEnter(JollyGameObject other)

public virtual void OnTriggerStay(JollyGameObject other)

public virtual void OnTriggerExit(JollyGameObject other)

public JollyCharacterController AddCharacterController()

public void Destroy()
```

Core/JollyGameTime.cs

```
public sealed class JollyGameTime

public double TotalSeconds { get; internal set; } public double DeltaSeconds { get; internal set; } public
double DeltaMilliseconds => DeltaSeconds * 1000.0;

public double FPS { get; internal set; } public double FrameTime => DeltaMilliseconds;

public long FrameCount { get; internal set; } } }
```

Core/JollySpriteObject.cs

```
public class JollySpriteObject : JollyGameObject

public JollySprite Sprite { get; } public bool IsColliding => _collisions.Count > 0;

public bool IsTriggered => _triggers.Count > 0;

public IReadOnlyCollection<JollyGameObject> Collisions => _collisions;
```

```
public IReadOnlyCollection<JollyGameObject> Triggers => _triggers;
public JollyCollisionLayer CollisionLayer => Collider?.Layer ?? JollyCollisionLayer.None;
public JollyCollisionLayer CollisionMask => Collider?.Mask ?? JollyCollisionLayer.None;
public JollySpriteObject(JollySprite sprite)
public override void Initialize()
public override void Update(JollyGameTime gameTime)
public override void Draw()
public override void Shutdown()
public void EnableCollision(JollyCollisionMode mode = JollyCollisionMode.Bounds, byte alphaThreshold = 1)
public void SetRectangleCollider(float width, float height)
public void SetCircleCollider(float radius)
public void SetPixelCollider(byte alphaThreshold = 1)
public void SetCollisionLayers(JollyCollisionLayer layer, JollyCollisionLayer mask)
public void SetTrigger(bool isTrigger = true)
public void DisableCollision()
public override void OnCollisionEnter(JollyGameObject other)
public override void OnCollisionStay(JollyGameObject other)
public override void OnCollisionExit(JollyGameObject other)
public override void OnTriggerEnter(JollyGameObject other)
public override void OnTriggerStay(JollyGameObject other)
public override void OnTriggerExit(JollyGameObject other)
```

Diagnostics/JollyDiagnosticScope.cs

```
public struct JollyDiagnosticScope : IDisposable
public void Dispose()
```

Diagnostics/JollyDiagnostics.cs

```
public sealed class JollyDiagnostics
public void BeginFrame()
public void EndFrame()
public JollyDiagnosticScope MeasureUpdate()
public JollyDiagnosticScope MeasureRender()
public void SetEntityCount(int count)
public void SetSpriteCount(int count)
public void SetTextureCount(int count)
public void SetRenderBatchCount(int count)
public void SetRenderDrawCallCount(int count)
public void SetRenderMaxBatchSize(int count)
public void SetConnectedClients(int count)
public void RecordNetworkMessageSent(int byteCount)
public void RecordNetworkMessageReceived(int byteCount)
public void SetNetworkLatency(double milliseconds)
public JollyProfile RegisterProfile(string name)
public JollyProfileScope Profile(JollyProfile? profile)
public JollyDiagnosticsSnapshot GetSnapshot()
public void Reset()
```

Diagnostics/JollyDiagnosticsFormatter.cs

```
public static class JollyDiagnosticsFormatter

public static string Format(JollyDiagnosticsSnapshot snapshot)
```

Diagnostics/JollyDiagnosticsSnapshot.cs

```
public sealed class JollyDiagnosticsSnapshot

public sealed class JollyProfileSnapshot

public string Name { get; init; } = string.Empty;

public long CallCount { get; init; } public double LastMilliseconds { get; init; } public double
TotalMilliseconds { get; init; } public double AverageMilliseconds { get; init; } public double
MaximumMilliseconds { get; init; } } }
```

Diagnostics/JollyDiagnosticsViewModel.cs

```
public sealed class JollyDiagnosticsViewModel : INotifyPropertyChanged

public event PropertyChangedEventHandler? PropertyChanged;

public double FPS

public double FrameTimeMilliseconds

public double UpdateTimeMilliseconds

public double RenderTimeMilliseconds

public long FrameCount

public long UpdateCount

public long RenderCount

public int EntityCount

public int SpriteCount

public int TextureCount

public int RenderBatchCount

public int RenderDrawCallCount

public int RenderMaxBatchSize

public int ConnectedClients

public long NetworkMessagesSent

public long NetworkMessagesReceived

public long NetworkBytesSent

public long NetworkBytesReceived

public double NetworkLatencyMilliseconds

public IReadOnlyDictionary<string, JollyProfileSnapshot> Profiles

public void Refresh(JollyDiagnostics diagnostics)

public string GetSummary()

public string GetRenderSummary()

public JollyProfileSnapshot? GetProfile(string name)

public long GetProfileCallCount(string name)
```

Diagnostics/JollyProfile.cs

```
public sealed class JollyProfile

public string Name { get; }  internal long CallCount;
```

Diagnostics/JollyProfileScope.cs

```
public struct JollyProfileScope : IDisposable

public void Dispose()
```

Graphics/JollyLineSnapshot.cs

```
public readonly struct JollyLineSnapshot(Vector2 start, Vector2 end, JollyColor color, float thickness, float
startThickness = 1.0f, float endThickness = 1.0f)

public Vector2 Start { get; } = start;

public Vector2 End { get; } = end;

public JollyColor Color { get; } = color;

public float Thickness { get; } = thickness;

public float StartThickness { get; } = startThickness;

public float EndThickness { get; } = endThickness;
```

Graphics/JollyRenderFrame.cs

```
public sealed class JollyRenderFrame

public IReadOnlyList<JollySpriteSnapshot> Sprites => _sprites;

public IReadOnlyList<JollyLineSnapshot> Lines => _lines;

public JollyRenderFrame(IEnumerable<JollySprite> sprites, IEnumerable<JollyLineSnapshot>? lines = null)
```

Graphics/JollyRenderTarget.cs

```
public sealed class JollyRenderTarget : IDisposable

public int Width { get; } public int Height { get; }  internal JollyRenderTarget(ID3D11Texture2D texture,
ID3D11RenderTargetView renderTargetView, ID3D11ShaderResourceView shaderResourceView, int width, int height)

public void Dispose()
```

Graphics/JollyRenderer.cs

```
public abstract class JollyRenderer

public abstract JollyTexture LoadTexture(string filePath);

public abstract JollySprite CreateSprite(JollyTexture texture);

public JollySprite CreateSprite(string filePath)
```

Graphics/JollySprite.cs

```
public sealed class JollySprite

public event Action<JollySprite, string>? AnimationCompleted;

public JollyCollisionMode CollisionMode { get; set; } = JollyCollisionMode.None;

public byte CollisionAlphaThreshold { get; set; } = 1;

public string Name { get; set; } = string.Empty;

public JollyTexture Texture { get; }  public JollyVector2 Position

public JollyVector2 Size

public JollyVector2 Scale

public float X

public float Y

public float Width

public float Height

public float Rotation

public float Opacity

public JollyColor Color { get; set; } = JollyColor.White;

public bool Visible

public int Layer

public bool FlipHorizontal

public bool FlipVertical

public float SourceX
```

```
public float SourceY

public float SourceWidth

public float SourceHeight

public bool AnimationPlaying => _animationPlaying;

public int AnimationFrame => _animationFrame;

public float AnimationFPS => _animationFPS;

public int AnimationFrameCount => _currentAnimation?.FrameCount ?? _animationSheet?.FrameCount ?? 0;

public bool AnimationLoop

public JollySpriteSheet? AnimationSheet => _animationSheet;

public string? CurrentAnimation => _currentAnimation?.Name;

public bool IsAnimationComplete

public IReadOnlyDictionary<string, JollyAnimation> Animations => _animations;

public void AddAnimation(JollyAnimation animation)

public bool RemoveAnimation(string name)

public bool HasAnimation(string name)

public JollyAnimation? GetAnimation(string name)

public void Play(string animationName)

public void SetAnimation(JollySpriteSheet sheet, float framesPerSecond, bool loop = true)

public void Play()

public void Pause()

public void Stop()

public void SetAnimationFrame(int frame)

public void SetFrame(JollySpriteSheetFrame frame, JollySpriteSheet sheet)
```

Graphics/JollySpriteSnapshot.cs

```
public readonly struct JollySpriteSnapshot

public string Name { get; } public JollyTexture Texture { get; } public float X { get; } public float Y { get;
} public float Width { get; } public float Height { get; } public float Rotation { get; } public float Opacity {
 get; } public JollyColor Color { get; init; } public bool Visible { get; } public int Layer { get; } public
bool FlipHorizontal { get; } public bool FlipVertical { get; } public float SourceX { get; } public float
SourceY { get; } public float SourceWidth { get; } public float SourceHeight { get; } public JollyVector2 Scale
{ get; }  internal JollySpriteSnapshot(JollySprite sprite)
```

Graphics/JollyTexture.cs

```
public sealed class JollyTexture

public int Width { get; } public int Height { get; }  internal object RendererTexture { get; }  internal byte[]
AlphaMask { get; }  internal JollyTexture(object rendererTexture, int width, int height, byte[] alphaMask)

public byte GetAlpha(int x, int y)
```

Input/JollyGamepad.cs

```
public sealed class JollyGamepad

public int Index { get; }  public bool IsConnected

public float LeftStickX

public float LeftStickY

public float RightStickX

public float RightStickY

public float LeftTrigger

public float RightTrigger

public JollyGamepad(int index)

public bool IsButtonDown(JollyGamepadButton button)

public bool IsButtonPressed(JollyGamepadButton button)
```

```
public bool IsButtonReleased(JollyGamepadButton button)
public uint PacketNumber;
public XInputGamepad Gamepad;
public ushort Buttons;
public byte LeftTrigger;
public byte RightTrigger;
public short ThumbLX;
public short ThumbLY;
public short ThumbRX;
public short ThumbRY;
```

Input/JollyGamepadButton.cs

```
public enum JollyGamepadButton
```

Input/JollyInput.cs

```
public sealed class JollyInput
public JollyKeyboard Keyboard => _keyboard;
public JollyMouse Mouse => _mouse;
public IReadOnlyList<JollyGamepad> Gamepads => _gamepads;
public void Attach(Window window)
public void Detach(Window window)
public bool IsKeyDown(Key key)
public bool IsKeyPressed(Key key)
public bool IsKeyReleased(Key key)
public double MouseX => _mouse.X;
public double MouseY => _mouse.Y;
public System.Windows.Point MousePosition => _mouse.Position;
public int MouseWheel => _mouse.Wheel;
public bool IsMouseButtonDown(JollyMouseButton button)
public bool IsMouseButtonPressed(JollyMouseButton button)
public bool IsMouseButtonReleased(JollyMouseButton button)
public Vector2 GetMouseWorldPosition(Camera2D camera)
```

Input/JollyKeyboard.cs

```
public sealed class JollyKeyboard
public void Attach(Window window)
public void Detach(Window window)
public bool IsKeyDown(Key key)
public bool IsKeyPressed(Key key)
public bool IsKeyReleased(Key key)
```

Input/JollyMouse.cs

```
public enum JollyMouseButton
public sealed class JollyMouse
public double X
public double Y
public System.Windows.Point Position
public int Wheel
public void Attach(Window window)
```

```
public void Detach(Window window)

public bool IsButtonDown(JollyMouseButton button)

public bool IsButtonPressed(JollyMouseButton button)

public bool IsButtonReleased(JollyMouseButton button)
```

Math/JollyColor.cs

```
public readonly struct JollyColor : IEquatable<JollyColor>

public float R { get; } public float G { get; } public float B { get; } public float A { get; }  public
JollyColor(float r, float g, float b, float a = 1.0f)

public static JollyColor White => new(1.0f, 1.0f, 1.0f, 1.0f);

public static JollyColor Black => new(0.0f, 0.0f, 0.0f, 1.0f);

public static JollyColor Red => new(1.0f, 0.0f, 0.0f, 1.0f);

public static JollyColor Green => new(0.0f, 1.0f, 0.0f, 1.0f);

public static JollyColor Blue => new(0.0f, 0.0f, 1.0f, 1.0f);

public static JollyColor Yellow => new(1.0f, 1.0f, 0.0f, 1.0f);

public static JollyColor Transparent => new(0.0f, 0.0f, 0.0f, 0.0f);

public static JollyColor FromBytes(byte r, byte g, byte b, byte a = 255)

public bool Equals(JollyColor other)

public override bool Equals(object? obj)

public override int GetHashCode()

public static bool operator ==(JollyColor left, JollyColor right)

public static bool operator !=(JollyColor left, JollyColor right)
```

Math/JollyVector2.cs

```
public struct JollyVector2

public float X;

public float Y;

public JollyVector2(float x, float y)

public static JollyVector2 Zero => new JollyVector2(0.0f, 0.0f);

public static JollyVector2 One => new JollyVector2(1.0f, 1.0f);

public static JollyVector2 operator +(JollyVector2 left, JollyVector2 right)

public static JollyVector2 operator -(JollyVector2 left, JollyVector2 right)

public static JollyVector2 operator *(JollyVector2 value, float scalar)

public static JollyVector2 operator /(JollyVector2 value, float scalar)

public float Length()

public float LengthSquared()

public void Normalize()

public static JollyVector2 Normalize(JollyVector2 value)
```

Networking/JollyNetworkClient.cs

```
public sealed class JollyNetworkClient : IDisposable

public bool IsConnected => _connection?.IsConnected == true;

public string Host { get; private set; } = string.Empty;

public int Port { get; private set; }  public event Action? Connected;

public event Action? Disconnected;

public event Action<JollyNetworkMessage>? MessageReceived;

public void Connect(string host, int port)

public async Task ConnectAsync(string host, int port, CancellationToken cancellationToken = default)

public void Send(JollyNetworkMessage message)
```

```
public void SendText(string text)
public void Disconnect()
public void Dispose()
```

Networking/JollyNetworkConnection.cs

```
public sealed class JollyNetworkConnection : IDisposable
public string Id { get; } = Guid.NewGuid().ToString("N");
public EndPoint? RemoteEndPoint { get; }  public bool IsConnected => !_disposed && _client.Connected;
public event Action<JollyNetworkConnection, JollyNetworkMessage>? MessageReceived;
public event Action<JollyNetworkConnection>? Disconnected;
public void Send(JollyNetworkMessage message)
public void SendText(string text)
public void Disconnect()
public void Dispose()
```

Networking/JollyNetworkMessage.cs

```
public sealed class JollyNetworkMessage
public JollyNetworkMessage(byte[] payload)
public byte[] Payload { get; }  public string Text => Encoding.UTF8.GetString(Payload);
public static JollyNetworkMessage FromText(string text)
```

Networking/JollyNetworkServer.cs

```
public sealed class JollyNetworkServer : IDisposable
public bool IsRunning => _listener != null;
public int Port { get; private set; }  public IReadOnlyList<JollyNetworkConnection> Clients
public event Action<JollyNetworkConnection>? ClientConnected;
public event Action<JollyNetworkConnection>? ClientDisconnected;
public event Action<JollyNetworkConnection, JollyNetworkMessage>? MessageReceived;
public void Start(int port)
public void Stop()
public void Broadcast(JollyNetworkMessage message)
public void BroadcastText(string text)
public void Dispose()
```

Particles/JollyParticle.cs

```
public sealed class JollyParticle
public System.Numerics.Vector2 Position { get; internal set; } public System.Numerics.Vector2 Velocity { get;
internal set; } public System.Numerics.Vector2 Acceleration { get; internal set; } public float Drag { get;
internal set; } public float Life { get; internal set; } public float MaximumLife { get; internal set; } public
float Size { get; internal set; } public float StartSize { get; internal set; } public float EndSize { get;
internal set; } public float Rotation { get; internal set; } public float RotationSpeed { get; internal set; }
public float Opacity { get; internal set; } public JollyColor Color { get; internal set; } = JollyColor.White;
public int Layer { get; internal set; } public bool IsAlive => Life > 0.0f;
public float NormalizedLife
public float Age
```

Particles/JollyParticleEmitter.cs

```
public sealed class JollyParticleEmitter
public Vector2 Position { get; set; } public bool Enabled { get; set; } = true;
public float ParticlesPerSecond { get; set; } = 20.0f;
public float MinimumLife { get; set; } = 0.5f;
```

```
public float MaximumLife { get; set; } = 1.0f;

public float MinimumSize { get; set; } = 4.0f;

public float MaximumSize { get; set; } = 8.0f;

public float MinimumSpeed { get; set; } = 20.0f;

public float MaximumSpeed { get; set; } = 60.0f;

public float Direction { get; set; } public float Spread { get; set; } = MathF.PI * 2.0f;

public Vector2 Gravity { get; set; } public float Drag { get; set; } public float MinimumRotationSpeed { get;
set; } public float MaximumRotationSpeed { get; set; } public float Opacity { get; set; } = 1.0f;

public JollyColor Color { get; internal set; } = JollyColor.White;

public int Layer { get; set; } public JollyParticleEmitterShape Shape { get; set; } = JollyParticleEmitterShape.
Point;

public float ShapeWidth { get; set; } = 32.0f;

public float ShapeHeight { get; set; } = 32.0f;

public float ShapeRadius { get; set; } = 16.0f;

public int Update(float deltaSeconds, List<JollyParticle> particles)

public void Emit(int count, List<JollyParticle> particles)

public void Reset()

public void Burst(int count, List<JollyParticle> particles)

public void ApplyPreset(JollyParticlePreset preset)
```

Particles/JollyParticleEmitterShape.cs

```
public enum JollyParticleEmitterShape
```

Particles/JollyParticlePreset.cs

```
public enum JollyParticlePreset
```

Particles/JollyParticleRenderer.cs

```
public sealed class JollyParticleRenderer

public IReadOnlyList<JollySprite> Sprites => _sprites;

public JollyParticleRenderer(JollyParticleSystem particleSystem, Func<JollyTexture, JollySprite> spriteFactory,
Func<JollyColor, JollyTexture> textureFactory)

public void Update(JollyTexture texture)

public void Clear()
```

Particles/JollyParticleSystem.cs

```
public sealed class JollyParticleSystem

public float Drag { get; internal set; }  public IReadOnlyList<JollyParticle> Particles => _particles;

public IReadOnlyList<JollyParticleEmitter> Emitters => _emitters;

public int ParticleCount => _particles.Count;

public void AddEmitter(JollyParticleEmitter emitter)

public void RemoveEmitter(JollyParticleEmitter emitter)

public void Update(float deltaSeconds)

public void Clear()

public void Emit(JollyParticleEmitter emitter, int count)
```

Physics/JollyCharacterController.cs

```
public sealed class JollyCharacterController

public JollyGameObject GameObject => _gameObject;

public float MoveAcceleration { get; set; } = 2200.0f;

public float MaxMoveSpeed { get; set; } = 300.0f;
```

```
public float GroundDeceleration { get; set; } = 2600.0f;

public float AirDeceleration { get; set; } = 800.0f;

public float JumpVelocity { get; set; } = 450.0f;

public float JumpGravity { get; set; } = 980.0f;

public float FallGravity { get; set; } = 1400.0f;

public float JumpCutGravity { get; set; } = 1800.0f;

public float MaxFallSpeed { get; set; } = 900.0f;

public float CoyoteTime { get; set; } = 0.10f;

public float JumpBufferTime { get; set; } = 0.10f;

public bool JumpHeld { get; private set; }  public bool IsGrounded => _gameObject.IsGrounded;

public JollyCharacterController(JollyGameObject gameObject)

public void Reset()

public void Move(float direction)

public void Stop()

public void SetJumpHeld(bool held)

public void RequestJump()

public void Update(JollyGameTime gameTime)
```

Physics/JollyCollider.cs

```
public sealed class JollyCollider

public JollySprite Sprite { get; } public bool Enabled { get; set; } = true;

public bool IsTrigger { get; set; } public JollyCollisionLayer Layer { get; set; } = JollyCollisionLayer.Layer0;

public JollyCollisionLayer Mask { get; set; } = JollyCollisionLayer.All;

public JollyColliderShape Shape { get; set; } = JollyColliderShape.Rectangle;

public float Width { get; set; } public float Height { get; set; } public float Radius { get; set; } public
byte AlphaThreshold { get; set; } = 1;

public JollyCollisionMode Mode

public JollyCollider(JollySprite sprite)

public bool CanCollideWith(JollyCollider other)

public bool CollidesWith(JollyCollider other)

public bool Check(JollyCollider other)
```

Physics/JollyColliderShape.cs

```
public enum JollyColliderShape
```

Physics/JollyCollisionDebug.cs

```
public static class JollyCollisionDebug

public static bool Enabled { get; set; }  public static bool LogPixelChecks { get; set; }  public static bool
LogPixelHits { get; set; } = true;

public static bool DrawColliders { get; set; }  public static void CollisionEnter(JollyGameObject objectA,
JollyGameObject objectB)

public static void CollisionStay(JollyGameObject objectA, JollyGameObject objectB)

public static void CollisionExit(JollyGameObject objectA, JollyGameObject objectB)

public static void TriggerEnter(JollyGameObject objectA, JollyGameObject objectB)

public static void TriggerStay(JollyGameObject objectA, JollyGameObject objectB)

public static void TriggerExit(JollyGameObject objectA, JollyGameObject objectB)

public static void PixelHit(JollySprite spriteA, JollySprite spriteB, int x, int y)
```

Physics/JollyCollisionLayer.cs

```
public enum JollyCollisionLayer : uint
```

Physics/JollyCollisionMode.cs

```
public enum JollyCollisionMode
```

Physics/JollyCollisionResult.cs

```
public readonly struct JollyCollisionResult

public JollySprite SpriteA { get; } public JollySprite SpriteB { get; } public bool Collided { get; }  public
JollyCollisionResult(JollySprite spriteA, JollySprite spriteB, bool collided)
```

Physics/JollyCollisionSystem.cs

```
public static class JollyCollisionSystem

public static bool Check(JollyCollider colliderA, JollyCollider colliderB)
```

Physics/JollyPhysicsSystem.cs

```
public static class JollyPhysicsSystem

public static void Update(JollyGameObject gameObject, JollyGameTime gameTime)

public static void ResolveTileMap(JollyGameObject gameObject, JollyTileMap tileMap, float tileMapX, float
tileMapY, float deltaTime)

public static bool Jump(JollyGameObject gameObject, float velocity)

public static void Resolve(JollyGameObject objectA, JollyGameObject objectB, float deltaTime)
```

Physics/JollyProjectileObject.cs

```
public sealed class JollyProjectileObject : JollySpriteObject

public float Lifetime { get; set; } = 5.0f;

public int Damage { get; set; } = 1;

public bool DestroyOnCollision { get; set; } = true;

public bool IsExpired { get; private set; }  public JollyProjectileObject(JollySprite sprite) : base(sprite)

public override void Initialize()

public override void Update(JollyGameTime gameTime)

public override void OnCollisionEnter(JollyGameObject other)
```

Rendering/Camera/Camera2D.cs

```
public sealed class Camera2D

public Vector2 Position

public float X

public float Y

public float Zoom

public float Rotation

public float FollowSpeed

public Vector2 ViewportSize { get; private set; } public bool BoundsEnabled => _boundsEnabled;

public float BoundsLeft => _boundsLeft;

public float BoundsTop => _boundsTop;

public float BoundsRight => _boundsRight;

public float BoundsBottom => _boundsBottom;

public bool IsShaking => _shakeTime > 0.0f;

public Vector2 ShakeOffset => _shakeOffset;

public void SetViewport(float width, float height)

public void SetBounds(float left, float top, float right, float bottom)

public void ClearBounds()

public void Follow(Vector2 targetPosition)
```

```
public void FollowSmooth(Vector2 targetPosition, float deltaSeconds)

public Vector2 WorldToScreen(Vector2 worldPosition)

public Vector2 ScreenToWorld(Vector2 screenPosition)

public Matrix3x2 GetWorldToScreenMatrix()

public CameraState CaptureState()

public void Shake(float intensity, float duration)

public void UpdateShake(float deltaSeconds)
```

Rendering/Camera/CameraState.cs

```
public readonly struct CameraState

public Vector2 Position { get; }  public float Zoom { get; }  public float Rotation { get; }  public Vector2
ViewportSize { get; }  public CameraState( Vector2 position, float zoom, float rotation, Vector2 viewportSize)

public Matrix3x2 GetWorldToScreenMatrix()

public Vector2 WorldToScreen(Vector2 worldPosition)

public Vector2 ScreenToWorld(Vector2 screenPosition)
```

Rendering/DirectX/DirectXControl.xaml.cs

```
public partial class DirectXControl : System.Windows.Controls.UserControl//, System.ComponentModel.
INotifyPropertyChanged

public event Action<double, double>? MouseMoved;

public event Action? MouseButtonDown;

public event Action? MouseButtonUp;

public event Action<float, float>? RenderSizeChanged;

public bool LineGlowEnabled

public float LineGlowWidth

public float LineGlowStrength

public float LineGlowFalloff

public JollyDiagnostics? Diagnostics { get; set; }  public DirectXControl()

public DirectXStretchMode StretchMode

public DirectXImageFilter ImageFilter

public double MaxFPS

public bool VSync

public bool ClearEveryFrame

public System.Windows.Media.Color ClearColor

public static readonly System.Windows.DependencyProperty FPSProperty = System.Windows.DependencyProperty.
Register(nameof(FPS), typeof(double), typeof(DirectXControl), new System.Windows.PropertyMetadata(0.0));

public double FPS

public static readonly System.Windows.DependencyProperty FrameTimeProperty = System.Windows.DependencyProperty.
Register(nameof(FrameTime), typeof(double), typeof(DirectXControl), new System.Windows.PropertyMetadata(0.0));

public double FrameTime

public bool IsDirectXReady => _renderer != null && _renderer.IsInitialized;

public event EventHandler<System.Windows.Point>? MouseScreenPositionChanged;

public async Task WaitForReadyAsync()

public void LoadImage(string filePath)

public void SetImage(BitmapSource source)

public void SetImage(byte[] pixels, int width, int height)

public void SetImage(DirectXImage image)

public void ClearScreen()

public void Clear()

public DirectXTexture CreateTexture(string filePath)
```

```
public DirectXTexture CreateTexture(DirectXImage image)

public DirectXJollyRenderer CreateJollyRenderer()
```

Rendering/DirectX/DirectXImage.cs

```
public sealed class DirectXImage

public int Width { get; } public int Height { get; } public ReadOnlyMemory<byte> Pixels { get; }  public
DirectXImage(int width, int height, ReadOnlyMemory<byte> pixels)

public static DirectXImage FromFile(string filePath)

public static DirectXImage FromBitmapSource(BitmapSource source)
```

Rendering/DirectX/DirectXRenderer.cs

```
public enum DirectXImageFilter

public enum DirectXStretchMode

public readonly struct SpriteVertex

public readonly float X;

public readonly float Y;

public readonly float Z;

public readonly float U;

public readonly float V;

public SpriteVertex(float x, float y, float z, float u, float v)

public struct SpriteConstants

public float PositionX;

public float PositionY;

public float Width;

public float Height;

public float Rotation;

public float Opacity;

public float ViewportWidth;

public float ViewportHeight;

public float FlipHorizontal;

public float FlipVertical;

public float SourceX;

public float SourceY;

public float SourceWidth;

public float SourceHeight;

public float Padding0;

public float Padding1;

public struct SpriteInstance

public float PositionX;

public float PositionY;

public float Width;

public float Height;

public float Rotation;

public float Opacity;

public float FlipHorizontal;

public float FlipVertical;

public float SourceX;

public float SourceY;
```

```
public float SourceWidth;
public float SourceHeight;
public float Padding0;
public float Padding1;
public float Padding2;
public float Padding3;
public struct LineVertex
public float X;
public float Y;
public float R;
public float G;
public float B;
public float A;
public float Edge;
public float CoreRatio;
public LineVertex(float x, float y, JollyColor color, float edge, float coreRatio)
public struct LineConstants
public float ViewportWidth;
public float ViewportHeight;
public float GlowStrength;
public float GlowFalloff;
public float GlowEnabled;
public float Padding0;
public float Padding1;
public float Padding2;
public sealed class DirectXRenderer : IDisposable
public double FPS => Volatile.Read(ref _fps);
public double FrameTime => Volatile.Read(ref _frameTime);
public bool IsInitialized => _initialized && !_disposed;
public int RenderSpriteCount => _renderSpriteCount;
public int RenderBatchCount => _renderBatchCount;
public int RenderDrawCallCount => _renderDrawCallCount;
public int RenderMaxBatchSize => _renderMaxBatchSize;
public int RenderInstanceBufferCapacity => _spriteInstanceCapacity;
public DirectXImageFilter ImageFilter { get; private set; } = DirectXImageFilter.Linear;
public DirectXStretchMode StretchMode { get; private set; } = DirectXStretchMode.Uniform;
public double MaxFPS
public bool VSync
public bool ClearEveryFrame
public MediaColor ClearColor
public bool LineGlowEnabled
public float LineGlowWidth
public float LineGlowStrength
public float LineGlowFalloff
public JollyDiagnostics? Diagnostics
public void Initialize(IntPtr hwnd, int width, int height)
public void StartRenderLoop()
```

```
public void StopRenderLoop()

public void SetStretchMode(DirectXStretchMode mode)

public void SetImageFilter(DirectXImageFilter filter)

public void LoadImage(string filePath)

public void SetImage(BitmapSource source)

public void SetImage(byte[] pixels, int width, int height)

public void SetImage(DirectXImage image)

public DirectXTexture CreateTextureResource(string filePath)

public DirectXTexture CreateTextureResource(DirectXImage image)

public void SubmitRenderFrame(JollyRenderFrame frame, Camera2D camera)

public void Clear()

public void ClearScreen()

public void Render()

public void Resize(int width, int height)

public readonly float X;

public readonly float Y;

public readonly float Z;

public readonly float U;

public readonly float V;

public ImageVertex(float x, float y, float z, float u, float v)

public void Dispose()
```

Rendering/DirectX/DirectXSprite.cs

```
public sealed class DirectXSprite

public DirectXTexture Texture { get; } public float X { get; set; } public float Y { get; set; } public float
Width { get; set; } public float Height { get; set; } public float Rotation { get; set; } public float Opacity {
 get; set; } = 1.0f;

public bool Visible { get; set; } = true;
```

Rendering/DirectX/DirectXSpriteBinding.cs

```
public JollySprite Sprite { get; }  public DirectXSpriteState State { get; }  public DirectXSpriteBinding(
JollySprite sprite, DirectXTexture texture)

public void Synchronize()
```

Rendering/DirectX/DirectXSpriteState.cs

```
public DirectXTexture Texture { get; } public float X;

public float Y;

public float Width;

public float Height;

public float Rotation;

public float Opacity;

public bool Visible;

public int Layer;

public DirectXSpriteState(DirectXTexture texture)
```

Rendering/DirectX/DirectXTexture.cs

```
public sealed class DirectXTexture

public DirectXImage Image { get; }  internal DirectXTexture(DirectXImage image)
```

Rendering/DirectX/JollyPhysicsDebugRenderer.cs

```
public sealed class JollyPhysicsDebugRenderer

public bool Enabled { get; set; } = true;

public bool TileMapEnabled { get; set; } = true;

public JollyPhysicsDebugRenderer(DirectXJollyRenderer renderer)

public IReadOnlyList<JollySprite> Sprites => _sprites;

public void Update(IReadOnlyList<JollyGameObject> objects)

public void Update(IReadOnlyList<JollyGameObject> objects, JollyTileMap? tileMap, float tileMapX, float
tileMapY)
```

Rendering/DirectXJollyRenderer.cs

```
public sealed class DirectXJollyRenderer : JollyRenderer

public Camera2D Camera { get; } = new();

public bool LineGlowEnabled

public float LineGlowWidth

public float LineGlowStrength

public float LineGlowFalloff

public JollyDiagnostics? Diagnostics

public System.Windows.Media.Color ClearColor

public DirectXJollyRenderer(DirectXControl control)

public override JollyTexture LoadTexture(string filePath)

public JollyTexture CreateSolidTexture(byte red, byte green, byte blue, byte alpha = 255)

public JollyTexture CreateCircleTexture(byte red, byte green, byte blue, byte alpha = 255, int size = 32)

public override JollySprite CreateSprite(JollyTexture texture)

public void SubmitFrame(JollyRenderFrame frame)

public JollyTexture CreateTextTexture(string text, string fontFamily, float fontSize, JollyColor color, bool
bold = false, bool italic = false)

public void Resize(float width, float height)
```

Rendering/JollyRenderSubmission.cs

```
public sealed class JollyRenderSubmission

public JollyRenderFrame Frame { get; }  public CameraState Camera { get; }  public JollyRenderSubmission(
JollyRenderFrame frame, Camera2D camera)
```

Rendering/JollySpriteSheet.cs

```
public enum JollySpriteSheetOrder

public readonly struct JollySpriteSheetFrame

public int Index { get; } public int Column { get; } public int Row { get; } public int X { get; } public int Y
{ get; } public int Width { get; } public int Height { get; }  internal JollySpriteSheetFrame(int index, int
column, int row, int x, int y, int width, int height)

public sealed class JollySpriteSheet

public JollyTexture Texture { get; } public int FrameWidth { get; } public int FrameHeight { get; } public int
Columns => _columns;

public int Rows => _rows;

public int FrameCount => _columns * _rows;

public JollySpriteSheetOrder Order { get; }  public JollySpriteSheet(JollyTexture texture, int frameWidth, int
frameHeight, JollySpriteSheetOrder order = JollySpriteSheetOrder.Horizontal)

public JollySpriteSheetFrame GetFrame(int index)
```

Rendering/JollyTextureManager.cs

```
public sealed class JollyTextureManager
```

```
public JollyTextureManager(JollyRenderer renderer)

public int Count => _textures.Count;

public JollyTexture Load(string filePath)

public bool TryGet(string filePath, out JollyTexture? texture)

public bool Contains(string filePath)

public void Clear()
```

Save/JollyGameObjectSaveState.cs

```
public sealed class JollyGameObjectSaveState

public string Id { get; set; } = string.Empty;

public string Name { get; set; } = string.Empty;

public string Tag { get; set; } = string.Empty;

public bool Active { get; set; } public JollyVector2 Position { get; set; } public JollyVector2 Scale { get;
set; } public float Rotation { get; set; } public bool Visible { get; set; } public JollyVector2 Velocity { get;
 set; } public JollyVector2 Acceleration { get; set; } public JollyVector2 Gravity { get; set; } public float
Drag { get; set; } public bool UsePhysics { get; set; } public bool UseGravity { get; set; } public bool
IsGrounded { get; set; } public bool WasGrounded { get; set; } public float GroundFriction { get; set; } public
float AirFriction { get; set; } public float Restitution { get; set; } public JollyColliderSaveState? Collider {
 get; set; }  public static JollyGameObjectSaveState Capture(JollyGameObject gameObject)

public void ApplyTo(JollyGameObject gameObject)

public sealed class JollyColliderSaveState

public bool Enabled { get; set; } public bool IsTrigger { get; set; } public JollyCollisionLayer Layer { get;
set; } public JollyCollisionLayer Mask { get; set; } public JollyColliderShape Shape { get; set; } public float
Width { get; set; } public float Height { get; set; } public float Radius { get; set; } public byte
AlphaThreshold { get; set; }  public static JollyColliderSaveState Capture(JollyCollider collider)

public void ApplyTo(JollyCollider collider)
```

Save/JollySaveManager.cs

```
public sealed class JollySaveManager

public const int CurrentFormatVersion = 1;

public string SaveDirectory { get; }  public JollySaveManager(string? saveDirectory = null)

public void SaveGame<T>(string slot, T state, int gameVersion = 1)

public T LoadGame<T>(string slot, int? expectedGameVersion = null)

public bool TryLoadGame<T>(string slot, out T? state, int? expectedGameVersion = null)

public bool Exists(string slot)

public bool HasBackup(string slot)

public bool RestoreBackup(string slot)

public void Delete(string slot)

public IReadOnlyList<string> GetSlots()

public int FormatVersion { get; set; } public int GameVersion { get; set; } public DateTimeOffset SavedUtc {
get; set; } public T? Payload { get; set; } } }  public sealed class JollySaveException : Exception

public JollySaveException(string message) : base(message)

public JollySaveException(string message, Exception innerException) : base(message, innerException)
```

Save/JollySceneSaveState.cs

```
public sealed class JollySceneSaveState

public string SceneName { get; set; } = string.Empty;

public List<JollyGameObjectSaveState> Objects { get; set; } = new();

public static JollySceneSaveState Capture(JollyScene scene)

public int ApplyTo(JollyScene scene)
```

Scene/JollyScene.cs

```
public abstract class JollyScene
```

```
public IReadOnlyList<JollyGameObject> Objects => _objects;

public int EntityCount => _objects.Count;

public string Name { get; set; } = string.Empty;

public bool IsInitialized => _initialized;

public JollyTileMap? TileMap => _tileMap;

public void SetTileMap(JollyTileMap tileMap, float x, float y)

public void ClearTileMap()

public virtual void OnEnter()

public virtual void OnExit()

public virtual void Initialize()

public void Add(JollyGameObject gameObject)

public void Spawn(JollyGameObject gameObject)

public void Remove(JollyGameObject gameObject)

public void Despawn(JollyGameObject gameObject)

public virtual void Update(JollyGameTime gameTime)

public virtual void Draw()

public IReadOnlyList<JollySprite> GetSprites()

public JollyGameObject? FindBySaveId(string saveId)

public JollyGameObject? Find(string name)

public T? Find<T>(string name) where T : JollyGameObject

public IReadOnlyList<T> FindAll<T>() where T : JollyGameObject

public bool Contains(JollyGameObject gameObject)

public virtual void Shutdown()

public static void ResolveTileMap(JollyGameObject gameObject, JollyTileMap tileMap, float tileMapX, float
tileMapY, float deltaTime)

public JollyGameObject ObjectA { get; }  public JollyGameObject ObjectB { get; }  public CollisionPair(
JollyGameObject objectA, JollyGameObject objectB)

public bool Equals(CollisionPair other)

public override bool Equals(object? obj)

public override int GetHashCode()
```

Scene/JollySceneManager.cs

```
public sealed class JollySceneManager

public JollyScene? CurrentScene { get; private set; } public JollyScene? PreviousScene { get; private set; }
public int EntityCount => CurrentScene?.EntityCount ?? 0;

public bool IsChangingScene => _pendingScenes.Count > 0;

public event Action<JollyScene?, JollyScene?>? SceneChanged;

public void ChangeScene(JollyScene scene)

public JollyGameObject? Find(string name)

public T? Find<T>(string name) where T : JollyGameObject

public IReadOnlyList<T> FindAll<T>() where T : JollyGameObject

public void Update(JollyGameTime gameTime)

public void Draw()

public void Shutdown()
```

UI/JollyUI9Slice.cs

```
public sealed class JollyUI9Slice

public JollyTexture Texture { get; } public float Left { get; set; } public float Top { get; set; } public
float Right { get; set; } public float Bottom { get; set; }  public JollyUI9Slice(JollyTexture texture, float
left, float top, float right, float bottom)
```

UI/JollyUIAnchor.cs

```
public enum JollyUIAnchor
```

UI/JollyUIAnimation.cs

```
public enum JollyUIAnimationProperty

public sealed class JollyUIAnimation

public JollyUIElement Element { get; } public JollyUIAnimationProperty Property { get; } public float Duration {
 get; } public bool IsComplete { get; private set; }  internal JollyUIAnimation(JollyUIElement element,
JollyUIAnimationProperty property, Vector2 start, Vector2 end, float duration)
```

UI/JollyUIButton.cs

```
public sealed class JollyUIButton : JollyUIElement

public string Text { get; set; } = "Button";

public JollyColor NormalColor { get; set; } = JollyColor.FromBytes(60, 60, 60, 255);

public JollyColor HoverColor { get; set; } = JollyColor.FromBytes(80, 80, 80, 255);

public JollyColor PressedColor { get; set; } = JollyColor.FromBytes(40, 40, 40, 255);

public JollyColor DisabledColor { get; set; } = JollyColor.FromBytes(35, 35, 35, 255);

public JollyColor TextColor { get; set; } = JollyColor.White;

public float FontSize { get; set; } = 18.0f;

public bool Bold { get; set; } public bool IsHovered { get; internal set; } public bool IsPressed { get;
internal set; }  public event Action<JollyUIButton>? Click;

public JollyColor CurrentColor

public JollyTexture? NormalTexture { get; set; } public JollyTexture? HoverTexture { get; set; } public
JollyTexture? PressedTexture { get; set; } public JollyTexture? DisabledTexture { get; set; } public
JollyTexture? CurrentTexture

public JollyUI9Slice? NormalNineSlice { get; set; } public JollyUI9Slice? HoverNineSlice { get; set; } public
JollyUI9Slice? PressedNineSlice { get; set; } public JollyUI9Slice? DisabledNineSlice { get; set; }  public
JollyUI9Slice? CurrentNineSlice
```

UI/JollyUIElement.cs

```
public class JollyUIElement

public JollyUIElement? Parent { get; private set; } public IReadOnlyList<JollyUIElement> Children => _children;

public Vector2 Position { get; set; } public Vector2 Size { get; set; } public bool Visible { get; set; } =
true;

public bool Enabled { get; set; } = true;

public int Layer { get; set; } public float Opacity { get; set; } = 1.0f;

public JollyUIAnchor Anchor { get; set; } = JollyUIAnchor.TopLeft;

public float MarginLeft { get; set; } public float MarginTop { get; set; } public float MarginRight { get; set;
} public float MarginBottom { get; set; }  public float X

public float Y

public float Width

public float Height

public float AbsoluteX

public float AbsoluteY

public Vector2 AbsolutePosition

public bool ContainsPoint(Vector2 point)

public void AddChild(JollyUIElement child)

public bool RemoveChild(JollyUIElement child)

public void RemoveAllChildren()

public void SetMargin(float left, float top, float right, float bottom)
```

UI/JollyUILayout.cs

```
public enum JollyUILayout
```

UI/JollyUIManager.cs

```
public sealed class JollyUIManager

public IReadOnlyList<JollyUIElement> Elements => _elements;

public System.Numerics.Vector2 MousePosition { get; private set; } public JollyUIElement? HoveredElement =>
_hoveredElement;

public JollyUIElement? PressedElement => _pressedElement; public float ScreenWidth { get; private set; }

public float ScreenHeight { get; private set; }  public void SetScreenSize(float width, float height)

public void Add(JollyUIElement element)

public bool Remove(JollyUIElement element)

public void Clear()

public JollyUIElement? HitTest(Vector2 point)

public void MouseMove(float x, float y)

public void MouseDown()

public void MouseUp()

public JollyUIAnimation AnimatePosition(JollyUIElement element, Vector2 target, float duration)

public JollyUIAnimation AnimateSize(JollyUIElement element, Vector2 target, float duration)

public JollyUIAnimation AnimateOpacity(JollyUIElement element, float target, float duration)

public void Update(float deltaSeconds)

public void UpdateLayout()
```

UI/JollyUIPanel.cs

```
public sealed class JollyUIPanel : JollyUIElement

public JollyColor Color { get; set; } = JollyColor.White;

public JollyColor BackgroundColor

public JollyUILayout Layout { get; set; } = JollyUILayout.None;

public float Spacing { get; set; } public float PaddingLeft { get; set; } public float PaddingTop { get; set; }
public float PaddingRight { get; set; } public float PaddingBottom { get; set; } public bool StretchChildren {
get; set; } } }
```

UI/JollyUIProgressBar.cs

```
public enum JollyUIProgressBarOrientation

public sealed class JollyUIProgressBar : JollyUIElement

public float Minimum

public float Maximum

public float Value

public float Percentage

public JollyUIProgressBarOrientation Orientation { get; set; } = JollyUIProgressBarOrientation.Horizontal;

public JollyColor BackgroundColor

public JollyColor FillColor

public JollyTexture? BackgroundTexture { get; set; }  public JollyTexture? FillTexture { get; set; }  private
bool _showText;

public bool ShowText

public string Text

public JollyColor TextColor

public float FontSize

public bool Bold

public bool TextDirty { get; internal set; } = true;
```

UI/JollyUIRenderer.cs

```
public sealed class JollyUIRenderer

public IReadOnlyList<JollySprite> Sprites => _sprites;

public JollyUIRenderer(JollyUIManager uiManager, Func<JollyTexture, JollySprite> spriteFactory, Func<byte, byte,
 byte, byte, JollyTexture> textureFactory, Func<string, string, float, JollyColor, bool, bool, JollyTexture>
textTextureFactory)

public void Update()

public void Clear()
```

UI/JollyUISlider.cs

```
public enum JollyUISliderOrientation

public sealed class JollyUISlider : JollyUIElement

public JollyUISliderOrientation Orientation { get; set; } = JollyUISliderOrientation.Horizontal;

public float Minimum

public float Maximum

public float Value

public float Percentage

public JollyColor TrackColor { get; set; } = JollyColor.FromBytes(50, 50, 50, 255);

public JollyColor FillColor { get; set; } = JollyColor.FromBytes(80, 160, 240, 255);

public JollyColor HandleColor { get; set; } = JollyColor.White;

public float TrackThickness { get; set; } = 8.0f;

public float HandleSize { get; set; } = 20.0f;

public JollyTexture? TrackTexture { get; set; }  public JollyTexture? FillTexture { get; set; }  public
JollyTexture? HandleTexture { get; set; }  public bool IsDragging

public event Action<JollyUISlider, float>? ValueChanged;
```

UI/JollyUIText.cs

```
public sealed class JollyUIText : JollyUIElement

public string Text

public string FontFamily

public float FontSize

public JollyColor Color

public bool Bold

public bool Italic

public bool Dirty { get; internal set; } = true;
```

World/JollyTileMap.cs

```
public sealed class JollyTileMap

public event Action<JollyGameObject, int, int, int>? HazardContact;

public JollyTexture? Texture { get; private set; } public int TextureColumns { get; private set; } public int
TextureRows { get; private set; } public int TextureTileCount => TextureColumns * TextureRows;

public int Width { get; } public int Height { get; } public float TileWidth { get; } public float TileHeight {
get; }  public JollyTileMap(int width, int height, float tileWidth, float tileHeight)

public void NotifyHazardContact(JollyGameObject gameObject, int tileX, int tileY, int tileIndex)

public bool IsSolid(int x, int y)

public void SetSolid(int x, int y, bool solid)

public int GetTile(int x, int y)

public void SetTile(int x, int y, int tileIndex)

public Vector2 GetTilePosition(int x, int y)

public Vector2 WorldToTile(float x, float y)
```

```
public bool IsSolidAtWorldPosition(float x, float y)

public bool TryGetTileAtWorldPosition(float x, float y, out int tileIndex)

public void SetTexture(JollyTexture texture)

public bool TryGetTileSourceRectangle(int tileIndex, out int x, out int y, out int width, out int height)

public void SetAnimatedTile(int tileIndex, int startFrame, int frameCount, float framesPerSecond, bool loop =
true)

public void RemoveAnimatedTile(int tileIndex)

public bool IsAnimatedTile(int tileIndex)

public void SetConveyorTile(int tileIndex, float speed)

public void RemoveConveyorTile(int tileIndex)

public bool IsConveyorTile(int tileIndex)

public float GetConveyorSpeed(int tileIndex)

public int GetAnimatedTileIndex(int tileIndex, float elapsedSeconds)

public void SetHazardTile(int tileIndex, bool hazard)

public bool IsHazardTile(int tileIndex)

public int StartFrame { get; } public int FrameCount { get; } public float FramesPerSecond { get; } public bool
Loop { get; }  public JollyTileAnimation(int startFrame, int frameCount, float framesPerSecond, bool loop)
```

World/JollyTileMapRenderer.cs

```
public sealed class JollyTileMapRenderer

public IReadOnlyList<JollySprite> Sprites => _sprites;

public int Layer { get; set; } public float X { get; set; } public float Y { get; set; }  public
JollyTileMapRenderer(JollyTileMap tileMap, Func<JollyTexture, JollySprite> spriteFactory)

public void Rebuild()

public void Update(Camera2D camera, float deltaSeconds)
```

Complete API Usage Reference

This reference complements `API_REFERENCE.md` by documenting **what the parameters mean**, expected ranges where the implementation enforces them, and the return/event role for every public method/constructor found in the supplied source.

`Animation/JollyAnimation.cs`

```
public string Name { get; } public JollySpriteSheet Sheet { get; } public int StartFrame { get; } public int FrameCount { get; } public double FPS { get; } public bool Loop { get; } public JollyAnimation(string name, JollySpriteSheet sheet, int startFrame, int frameCount, double fps, bool loop = true)
```

Parameter	Meaning
`name`	name used to identify the object, asset, animation, sound or slot.
`sheet`	sprite sheet supplying animation frames.
`startFrame`	first sprite-sheet frame used by the animation.
`frameCount`	number of frames used by the animation.
`fps`	frames/updates per second.
`loop`	whether playback loops.

Return: Returns `string` as declared by the API.

```
public JollySpriteSheetFrame GetFrame(int animationFrame)
```

Parameter	Meaning
`animationFrame`	zero-based frame index within the animation.

Return: Returns `JollySpriteSheetFrame` as declared by the API.

`Assets/JollyAssetHandle.cs`

```
public void Dispose()
```

Return: No value is returned.

`Assets/JollyAssetManager.cs`

```
public JollyAssetManager(DirectXJollyRenderer renderer, JollyAudioManager? audio = null, string? rootPath = null)
```

Parameter	Meaning
`renderer`	the renderer used to create or submit graphics resources.
`audio`	value supplied to the operation.
`rootPath`	root directory containing the engine asset folders.

Return: Returns `JollyAssetManager` as declared by the API.

```
public string RootPath { get; } public string TexturesPath => Path.Combine(RootPath, TexturesFolder);
```

Parameter	Meaning
`RootPath`	root directory containing the engine asset folders.
`TexturesFolder`	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public string AudioPath => Path.Combine(RootPath, AudioFolder);
```

Parameter	Meaning
`RootPath`	root directory containing the engine asset folders.
`AudioFolder`	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public string FontsPath => Path.Combine(RootPath, FontsFolder);
```

Parameter	Meaning
`RootPath`	root directory containing the engine asset folders.
`FontsFolder`	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public string MapsPath => Path.Combine(RootPath, MapsFolder);
```

Parameter	Meaning
`RootPath`	root directory containing the engine asset folders.
`MapsFolder`	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public string AnimationsPath => Path.Combine(RootPath, AnimationsFolder);
```

Parameter	Meaning
`RootPath`	root directory containing the engine asset folders.
`AnimationsFolder`	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public JollyTexture LoadTexture(string assetName)
```

Parameter	Meaning
`assetName`	value supplied to the operation.

Return: Returns `JollyTexture` as declared by the API.

```
public async Task<JollyTexture> LoadTextureAsync(string assetName, CancellationToken cancellationToken =
default)
```

Parameter	Meaning
`assetName`	value supplied to the operation.
`cancellationToken`	token used to cancel the asynchronous operation.

Return: Returns `async` as declared by the API.

```
public JollyAssetHandle<JollyTexture> AcquireTexture(string assetName)
```

Parameter	Meaning
`assetName`	value supplied to the operation.

Return: Returns `JollyAssetHandle<JollyTexture>` as declared by the API.

```
public void ReleaseTexture(string assetName)
```

Parameter	Meaning
assetName	value supplied to the operation.

Return: No value is returned.

```
public bool IsTextureLoaded(string assetName)
```

Parameter	Meaning
assetName	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void LoadSound(string assetName)
```

Parameter	Meaning
assetName	value supplied to the operation.

Return: No value is returned.

```
public void PlaySound(string assetName)
```

Parameter	Meaning
assetName	value supplied to the operation.

Return: No value is returned.

```
public void PlayMusic(string assetName, bool loop = true)
```

Parameter	Meaning
assetName	value supplied to the operation.
loop	whether playback loops.

Return: No value is returned.

```
public void UnloadSound(string assetName)
```

Parameter	Meaning
assetName	value supplied to the operation.

Return: No value is returned.

```
public string GetTexturePath(string assetName) => ResolveAssetPath(TexturesFolder,
NormalizeAssetName(assetName));
```

Parameter	Meaning
assetName)	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public string GetAudioPath(string assetName) => ResolveAssetPath(AudioFolder,
NormalizeAssetName(assetName));
```

Parameter	Meaning
assetName)	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public string GetFontPath(string assetName) => ResolveAssetPath(FontsFolder,
NormalizeAssetName(assetName));
```

Parameter	Meaning
assetName)	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public string GetMapPath(string assetName) => ResolveAssetPath(MapsFolder,
NormalizeAssetName(assetName));
```

Parameter	Meaning
assetName)	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public string GetAnimationPath(string assetName) => ResolveAssetPath(AnimationsFolder,
NormalizeAssetName(assetName));
```

Parameter	Meaning
assetName)	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public bool AssetExists(string category, string assetName)
```

Parameter	Meaning
category	asset category/folder name.
assetName	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void Validate(params JollyAssetRequirement[] requirements)
```

Parameter	Meaning
requirements	asset requirements that must exist.

Return: No value is returned.

```
public void Dispose()
```

Return: No value is returned.

```
public TextureEntry(JollyTexture texture)
```

Parameter	Meaning
texture	the texture used by the sprite or render operation.

Return: Returns `TextureEntry` as declared by the API.

```
public JollyTexture Texture { get; } public int ReferenceCount { get; set; } } } public sealed record JollyAssetRequirement(string Category, string Name);
```

Parameter	Meaning
Category	asset category/folder name.
Name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `type` as declared by the API.

`Audio/JollyAudioManager.cs`

```
public void LoadSound(string name, string path)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.
path	file-system path for the resource.

Return: No value is returned.

```
public void UnloadSound(string name)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: No value is returned.

```
public void PlaySound(string name)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: No value is returned.

```
public void StopAllSounds()
```

Return: No value is returned.

```
public void PlayMusic(string path, bool loop = true)
```

Parameter	Meaning
path	file-system path for the resource.
loop	whether playback loops.

Return: No value is returned.

```
public void PauseMusic()
```

Return: No value is returned.

```
public void ResumeMusic()
```

Return: No value is returned.

```
public void StopMusic()
```

Return: No value is returned.

```
public void ClearSounds()
```

Return: No value is returned.

```
public void Dispose()
```

Return: No value is returned.

```
public CachedSound Sound { get; } public WaveOut Output { get; } public SoundInstance(CachedSound sound, float volume)
```

Parameter	Meaning
sound	value supplied to the operation.
volume	value supplied to the operation.

Return: Returns `CachedSound` as declared by the API.

```
public void Dispose()
```

Return: No value is returned.

```
public WaveFormat WaveFormat { get; } public float[] AudioData { get; } public CachedSound(string audioFileName)
```

Parameter	Meaning
audioFileName	value supplied to the operation.

Return: Returns `WaveFormat` as declared by the API.

```
public CachedSoundSampleProvider(CachedSound sound)
```

Parameter	Meaning
sound	value supplied to the operation.

Return: Returns `CachedSoundSampleProvider` as declared by the API.

```
public int Read(Span<float> buffer)
```

Parameter	Meaning
buffer	value supplied to the operation.

Return: Returns `int` as declared by the API.

```
public LoopStream(WaveStream source)
```

Parameter	Meaning
source	source image/stream/resource.

Return: Returns `LoopStream` as declared by the API.

```
public override int Read(byte[] buffer, int offset, int count)
```

Parameter	Meaning
buffer	value supplied to the operation.
offset	value supplied to the operation.
count	number of items/frames to process.

Return: Returns `override` as declared by the API.

`Core/JollyGame.cs`

```
public JollySceneManager Scenes { get; } public JollyInput Input { get; } public JollyGameTime GameTime { get; } = new JollyGameTime();
```

Return: Returns `JollySceneManager` as declared by the API.

```
public double GameLoopActualFPS => Volatile.Read(ref _gameLoopActualFPS);
```

Parameter	Meaning
`_gameLoopActualFPS`	value supplied to the operation.

Return: Returns `double` as declared by the API.

```
public void Start()
```

Return: No value is returned.

```
public void Stop()
```

Return: No value is returned.

`Core/JollyGameObject.cs`

```
public string SaveId { get; set; } = Guid.NewGuid().ToString("N");
```

Parameter	Meaning
`).ToString("N"`	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public bool IsDestroyed { get; private set; } internal JollyScene? Scene { get; set; } public virtual void Initialize()
```

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public virtual void Update(JollyGameTime gameTime)
```

Parameter	Meaning
`gameTime`	the current game-time/update information.

Return: Returns `virtual` as declared by the API.

```
public virtual void Draw()
```

Return: Returns `virtual` as declared by the API.

```
public virtual void Shutdown()
```

Return: Returns `virtual` as declared by the API.

```
public void SetCollider(JollyCollider collider)
```

Parameter	Meaning
`collider`	value supplied to the operation.

Return: No value is returned.

```
public void RemoveCollider()
```

Return: No value is returned.

```
public virtual void OnCollisionEnter(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `virtual` as declared by the API.

```
public virtual void OnCollisionStay(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `virtual` as declared by the API.

```
public virtual void OnCollisionExit(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `virtual` as declared by the API.

```
public virtual void OnTriggerEnter(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `virtual` as declared by the API.

```
public virtual void OnTriggerStay(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `virtual` as declared by the API.

```
public virtual void OnTriggerExit(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `virtual` as declared by the API.

```
public JollyCharacterController AddCharacterController()
```

Return: Returns `JollyCharacterController` as declared by the API.

```
public void Destroy()
```

Return: No value is returned.

`Core/JollySpriteObject.cs`

```
public JollySpriteObject(JollySprite sprite)
```

Parameter	Meaning
sprite	value supplied to the operation.

Return: Returns `JollySpriteObject` as declared by the API.

```
public override void Initialize()
```

Return: Returns `override` as declared by the API.

```
public override void Update(JollyGameTime gameTime)
```

Parameter	Meaning
`gameTime`	the current game-time/update information.

Return: Returns `override` as declared by the API.

```
public override void Draw()
```

Return: Returns `override` as declared by the API.

```
public override void Shutdown()
```

Return: Returns `override` as declared by the API.

```
public void EnableCollision(JollyCollisionMode mode = JollyCollisionMode.Bounds, byte alphaThreshold = 1)
```

Parameter	Meaning
`mode`	operation or collision mode to use.
`alphaThreshold`	minimum alpha value used by pixel collision.

Return: No value is returned.

```
public void SetRectangleCollider(float width, float height)
```

Parameter	Meaning
`width`	width in engine/world pixels or source pixels, depending on the API.
`height`	height in engine/world pixels or source pixels, depending on the API.

Return: No value is returned.

```
public void SetCircleCollider(float radius)
```

Parameter	Meaning
`radius`	circle radius in world units.

Return: No value is returned.

```
public void SetPixelCollider(byte alphaThreshold = 1)
```

Parameter	Meaning
`alphaThreshold`	minimum alpha value used by pixel collision.

Return: No value is returned.

```
public void SetCollisionLayers(JollyCollisionLayer layer, JollyCollisionLayer mask)
```

Parameter	Meaning
`layer`	collision layer assigned to the collider.
`mask`	collision layers this collider accepts.

Return: No value is returned.

```
public void SetTrigger(bool isTrigger = true)
```

Parameter	Meaning
isTrigger	value supplied to the operation.

Return: No value is returned.

```
public void DisableCollision()
```

Return: No value is returned.

```
public override void OnCollisionEnter(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `override` as declared by the API.

```
public override void OnCollisionStay(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `override` as declared by the API.

```
public override void OnCollisionExit(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `override` as declared by the API.

```
public override void OnTriggerEnter(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `override` as declared by the API.

```
public override void OnTriggerStay(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `override` as declared by the API.

```
public override void OnTriggerExit(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `override` as declared by the API.

```
Diagnostics/JollyDiagnosticScope.cs
```

```
public void Dispose()
```

Return: No value is returned.

`Diagnostics/JollyDiagnostics.cs`

`public void BeginFrame()`

Return: No value is returned.

`public void EndFrame()`

Return: No value is returned.

`public JollyDiagnosticScope MeasureUpdate()`

Return: Returns `JollyDiagnosticScope` as declared by the API.

`public JollyDiagnosticScope MeasureRender()`

Return: Returns `JollyDiagnosticScope` as declared by the API.

`public void SetEntityCount(int count)`

Parameter	Meaning
count	number of items/frames to process.

Return: No value is returned.

`public void SetSpriteCount(int count)`

Parameter	Meaning
count	number of items/frames to process.

Return: No value is returned.

`public void SetTextureCount(int count)`

Parameter	Meaning
count	number of items/frames to process.

Return: No value is returned.

`public void SetRenderBatchCount(int count)`

Parameter	Meaning
count	number of items/frames to process.

Return: No value is returned.

`public void SetRenderDrawCallCount(int count)`

Parameter	Meaning
count	number of items/frames to process.

Return: No value is returned.

`public void SetRenderMaxBatchSize(int count)`

Parameter	Meaning
count	number of items/frames to process.

Return: No value is returned.

```
public void SetConnectedClients(int count)
```

Parameter	Meaning
count	number of items/frames to process.

Return: No value is returned.

```
public void RecordNetworkMessageSent(int byteCount)
```

Parameter	Meaning
byteCount	value supplied to the operation.

Return: No value is returned.

```
public void RecordNetworkMessageReceived(int byteCount)
```

Parameter	Meaning
byteCount	value supplied to the operation.

Return: No value is returned.

```
public void SetNetworkLatency(double milliseconds)
```

Parameter	Meaning
milliseconds	value supplied to the operation.

Return: No value is returned.

```
public JollyProfile RegisterProfile(string name)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `JollyProfile` as declared by the API.

```
public JollyProfileScope Profile(JollyProfile? profile)
```

Parameter	Meaning
profile	value supplied to the operation.

Return: Returns `JollyProfileScope` as declared by the API.

```
public JollyDiagnosticsSnapshot GetSnapshot()
```

Return: Returns `JollyDiagnosticsSnapshot` as declared by the API.

```
public void Reset()
```

Return: No value is returned.

`Diagnostics/JollyDiagnosticsFormatter.cs`

```
public static string Format(JollyDiagnosticsSnapshot snapshot)
```

Parameter	Meaning
snapshot	value supplied to the operation.

Return: Returns `string` as declared by the API.

`Diagnostics/JollyDiagnosticsSnapshot.cs`

```
public double FPS { get; init; } public double FrameTimeMilliseconds { get; init; } public double UpdateTimeMilliseconds { get; init; } public double RenderTimeMilliseconds { get; init; } public long FrameCount { get; init; } public long UpdateCount { get; init; } public long RenderCount { get; init; } public int EntityCount { get; init; } public int SpriteCount { get; init; } public int TextureCount { get; init; } public int RenderBatchCount { get; init; } public int RenderDrawCallCount { get; init; } public int RenderMaxBatchSize { get; init; } public int ConnectedClients { get; init; } public long NetworkMessagesSent { get; init; } public long NetworkMessagesReceived { get; init; } public long NetworkBytesSent { get; init; } public long NetworkBytesReceived { get; init; } public double NetworkLatencyMilliseconds { get; init; } public IReadOnlyDictionary<string, JollyProfileSnapshot> Profiles { get; init; } = new Dictionary<string, JollyProfileSnapshot>();
```

Return: Returns `double` as declared by the API.

`Diagnostics/JollyDiagnosticsViewModel.cs`

```
public void Refresh(JollyDiagnostics diagnostics)
```

Parameter	Meaning
`diagnostics`	value supplied to the operation.

Return: No value is returned.

```
public string GetSummary()
```

Return: Returns `string` as declared by the API.

```
public string GetRenderSummary()
```

Return: Returns `string` as declared by the API.

```
public JollyProfileSnapshot? GetProfile(string name)
```

Parameter	Meaning
`name`	name used to identify the object, asset, animation, sound or slot.

Return: Returns `JollyProfileSnapshot?` as declared by the API.

```
public long GetProfileCallCount(string name)
```

Parameter	Meaning
`name`	name used to identify the object, asset, animation, sound or slot.

Return: Returns `long` as declared by the API.

`Diagnostics/JollyProfileScope.cs`

```
public void Dispose()
```

Return: No value is returned.

`Graphics/JollyLineSnapshot.cs`

```
public readonly struct JollyLineSnapshot(Vector2 start, Vector2 end, JollyColor color, float thickness, float startThickness = 1.0f, float endThickness = 1.0f)
```

Parameter	Meaning
`start`	line start position.
`end`	line end position.
`color`	colour applied to the resource.
`thickness`	line thickness.
`startThickness`	line thickness at the start of the line.
`endThickness`	line thickness at the end of the line.

Return: Returns `type` as declared by the API.

`Graphics/JollyRenderFrame.cs`

```
public JollyRenderFrame(IEnumerable<JollySprite> sprites, IEnumerable<JollyLineSnapshot>? lines = null)
```

Parameter	Meaning
`sprites`	value supplied to the operation.
`lines`	value supplied to the operation.

Return: Returns `JollyRenderFrame` as declared by the API.

`Graphics/JollyRenderTarget.cs`

```
public int Width { get; } public int Height { get; } internal JollyRenderTarget(ID3D11Texture2D texture, ID3D11RenderTargetView renderTargetView, ID3D11ShaderResourceView shaderResourceView, int width, int height)
```

Parameter	Meaning
`texture`	the texture used by the sprite or render operation.
`renderTargetView`	value supplied to the operation.
`shaderResourceView`	value supplied to the operation.
`width`	width in engine/world pixels or source pixels, depending on the API.
`height`	height in engine/world pixels or source pixels, depending on the API.

Return: Returns `int` as declared by the API.

```
public void Dispose()
```

Return: No value is returned.

`Graphics/JollyRenderer.cs`

```
public abstract JollyTexture LoadTexture(string filePath);
```

Parameter	Meaning
`filePath`	path to the source file.

Return: Returns `JollyTexture` as declared by the API.

```
public abstract JollySprite CreateSprite(JollyTexture texture);
```

Parameter	Meaning
texture	the texture used by the sprite or render operation.

Return: Returns `JollySprite` as declared by the API.

```
public JollySprite CreateSprite(string filePath)
```

Parameter	Meaning
filePath	path to the source file.

Return: Returns `JollySprite` as declared by the API.

`Graphics/JollySprite.cs`

```
public void AddAnimation(JollyAnimation animation)
```

Parameter	Meaning
animation	value supplied to the operation.

Return: No value is returned.

```
public bool RemoveAnimation(string name)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool HasAnimation(string name)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public JollyAnimation? GetAnimation(string name)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `JollyAnimation?` as declared by the API.

```
public void Play(string animationName)
```

Parameter	Meaning
animationName	value supplied to the operation.

Return: No value is returned.

```
public void SetAnimation(JollySpriteSheet sheet, float framesPerSecond, bool loop = true)
```

Parameter	Meaning
`sheet`	sprite sheet supplying animation frames.
`framesPerSecond`	value supplied to the operation.
`loop`	whether playback loops.

Return: No value is returned.

```
public void Play()
```

Return: No value is returned.

```
public void Pause()
```

Return: No value is returned.

```
public void Stop()
```

Return: No value is returned.

```
public void SetAnimationFrame(int frame)
```

Parameter	Meaning
`frame`	value supplied to the operation.

Return: No value is returned.

```
public void SetFrame(JollySpriteSheetFrame frame, JollySpriteSheet sheet)
```

Parameter	Meaning
`frame`	value supplied to the operation.
`sheet`	sprite sheet supplying animation frames.

Return: No value is returned.

`Graphics/JollySpriteSnapshot.cs`

```
public string Name { get; } public JollyTexture Texture { get; } public float X { get; } public float Y { get; } public float Width { get; } public float Height { get; } public float Rotation { get; } public float Opacity { get; } public JollyColor Color { get; init; } public bool Visible { get; } public int Layer { get; } public bool FlipHorizontal { get; } public bool FlipVertical { get; } public float SourceX { get; } public float SourceY { get; } public float SourceWidth { get; } public float SourceHeight { get; } public JollyVector2 Scale { get; } internal JollySpriteSnapshot(JollySprite sprite)
```

Parameter	Meaning
`sprite`	value supplied to the operation.

Return: Returns `string` as declared by the API.

`Graphics/JollyTexture.cs`

```
public int Width { get; } public int Height { get; } internal object RendererTexture { get; } internal byte[] AlphaMask { get; } internal JollyTexture(object rendererTexture, int width, int height, byte[] alphaMask)
```

Parameter	Meaning
`rendererTexture`	value supplied to the operation.

Parameter	Meaning
`width`	width in engine/world pixels or source pixels, depending on the API.
`height`	height in engine/world pixels or source pixels, depending on the API.
`alphaMask`	value supplied to the operation.

Return: Returns `int` as declared by the API.

```
public byte GetAlpha(int x, int y)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: Returns `byte` as declared by the API.

`Input/JollyGamepad.cs`

```
public JollyGamepad(int index)
```

Parameter	Meaning
`index`	zero-based index.

Return: Returns `JollyGamepad` as declared by the API.

```
public bool IsButtonDown(JollyGamepadButton button)
```

Parameter	Meaning
`button`	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsButtonPressed(JollyGamepadButton button)
```

Parameter	Meaning
`button`	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsButtonReleased(JollyGamepadButton button)
```

Parameter	Meaning
`button`	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

`Input/JollyInput.cs`

```
public void Attach(Window window)
```

Parameter	Meaning
`window`	value supplied to the operation.

Return: No value is returned.

```
public void Detach(Window window)
```

Parameter	Meaning
window	value supplied to the operation.

Return: No value is returned.

```
public bool IsKeyDown(Key key)
```

Parameter	Meaning
key	keyboard key being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsKeyPressed(Key key)
```

Parameter	Meaning
key	keyboard key being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsKeyReleased(Key key)
```

Parameter	Meaning
key	keyboard key being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsMouseButtonDown(JollyMouseButton button)
```

Parameter	Meaning
button	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsMouseButtonPressed(JollyMouseButton button)
```

Parameter	Meaning
button	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsMouseButtonReleased(JollyMouseButton button)
```

Parameter	Meaning
button	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public Vector2 GetMouseWorldPosition(Camera2D camera)
```

Parameter	Meaning
camera	camera used for coordinate conversion.

Return: Returns `Vector2` as declared by the API.

`Input/JollyKeyboard.cs`

`public void Attach(Window window)`

Parameter	Meaning
`window`	value supplied to the operation.

Return: No value is returned.

`public void Detach(Window window)`

Parameter	Meaning
`window`	value supplied to the operation.

Return: No value is returned.

`public bool IsKeyDown(Key key)`

Parameter	Meaning
`key`	keyboard key being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

`public bool IsKeyPressed(Key key)`

Parameter	Meaning
`key`	keyboard key being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

`public bool IsKeyReleased(Key key)`

Parameter	Meaning
`key`	keyboard key being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

`Input/JollyMouse.cs`

`public void Attach(Window window)`

Parameter	Meaning
`window`	value supplied to the operation.

Return: No value is returned.

`public void Detach(Window window)`

Parameter	Meaning
`window`	value supplied to the operation.

Return: No value is returned.

`public bool IsButtonDown(JollyMouseButton button)`

Parameter	Meaning
button	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsButtonPressed(JollyMouseButton button)
```

Parameter	Meaning
button	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool IsButtonReleased(JollyMouseButton button)
```

Parameter	Meaning
button	mouse/gamepad button being queried.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
Math/JollyColor.cs
```

```
public float R { get; } public float G { get; } public float B { get; } public float A { get; } public JollyColor(float r, float g, float b, float a = 1.0f)
```

Parameter	Meaning
r	red component.
g	green component.
b	blue component.
a	alpha component.

Return: Returns `float` as declared by the API.

```
public static JollyColor White => new(1.0f, 1.0f, 1.0f, 1.0f);
```

Parameter	Meaning
1.0f	value supplied to the operation.
1.0f	value supplied to the operation.
1.0f	value supplied to the operation.
1.0f	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public static JollyColor Black => new(0.0f, 0.0f, 0.0f, 1.0f);
```

Parameter	Meaning
0.0f	value supplied to the operation.
0.0f	value supplied to the operation.
0.0f	value supplied to the operation.
1.0f	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public static JollyColor Red => new(1.0f, 0.0f, 0.0f, 1.0f);
```

Parameter	Meaning
1.0f	value supplied to the operation.
0.0f	value supplied to the operation.
0.0f	value supplied to the operation.
1.0f	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public static JollyColor Green => new(0.0f, 1.0f, 0.0f, 1.0f);
```

Parameter	Meaning
0.0f	value supplied to the operation.
1.0f	value supplied to the operation.
0.0f	value supplied to the operation.
1.0f	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public static JollyColor Blue => new(0.0f, 0.0f, 1.0f, 1.0f);
```

Parameter	Meaning
0.0f	value supplied to the operation.
0.0f	value supplied to the operation.
1.0f	value supplied to the operation.
1.0f	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public static JollyColor Yellow => new(1.0f, 1.0f, 0.0f, 1.0f);
```

Parameter	Meaning
1.0f	value supplied to the operation.
1.0f	value supplied to the operation.
0.0f	value supplied to the operation.
1.0f	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public static JollyColor Transparent => new(0.0f, 0.0f, 0.0f, 0.0f);
```

Parameter	Meaning
0.0f	value supplied to the operation.
0.0f	value supplied to the operation.
0.0f	value supplied to the operation.
0.0f	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public static JollyColor FromBytes(byte r, byte g, byte b, byte a = 255)
```

Parameter	Meaning
r	red component.
g	green component.
b	blue component.
a	alpha component.

Return: Returns `JollyColor` as declared by the API.

```
public bool Equals(JollyColor other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public override bool Equals(object? obj)
```

Parameter	Meaning
obj	value supplied to the operation.

Return: Returns `override` as declared by the API.

```
public override int GetHashCode()
```

Return: Returns `override` as declared by the API.

```
public static bool operator ==(JollyColor left, JollyColor right)
```

Parameter	Meaning
left	left world bound.
right	right world bound.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public static bool operator !=(JollyColor left, JollyColor right)
```

Parameter	Meaning
left	left world bound.
right	right world bound.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
Math/JollyVector2.cs
```

```
public JollyVector2(float x, float y)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: Returns `JollyVector2` as declared by the API.

```
public static JollyVector2 Zero => new JollyVector2(0.0f, 0.0f);
```

Parameter	Meaning
`0.0f`	value supplied to the operation.
`0.0f`	value supplied to the operation.

Return: Returns `JollyVector2` as declared by the API.

```
public static JollyVector2 One => new JollyVector2(1.0f, 1.0f);
```

Parameter	Meaning
`1.0f`	value supplied to the operation.
`1.0f`	value supplied to the operation.

Return: Returns `JollyVector2` as declared by the API.

```
public static JollyVector2 operator +(JollyVector2 left, JollyVector2 right)
```

Parameter	Meaning
`left`	left world bound.
`right`	right world bound.

Return: Returns `JollyVector2` as declared by the API.

```
public static JollyVector2 operator -(JollyVector2 left, JollyVector2 right)
```

Parameter	Meaning
`left`	left world bound.
`right`	right world bound.

Return: Returns `JollyVector2` as declared by the API.

```
public static JollyVector2 operator *(JollyVector2 value, float scalar)
```

Parameter	Meaning
`value`	value being assigned or changed.
`scalar`	value supplied to the operation.

Return: Returns `JollyVector2` as declared by the API.

```
public static JollyVector2 operator /(JollyVector2 value, float scalar)
```

Parameter	Meaning
`value`	value being assigned or changed.
`scalar`	value supplied to the operation.

Return: Returns `JollyVector2` as declared by the API.

```
public float Length()
```

Return: Returns `float` as declared by the API.

```
public float LengthSquared()
```

Return: Returns `float` as declared by the API.

```
public void Normalize()
```

Return: No value is returned.

```
public static JollyVector2 Normalize(JollyVector2 value)
```

Parameter	Meaning
value	value being assigned or changed.

Return: Returns `JollyVector2` as declared by the API.

`Networking/JollyNetworkClient.cs`

```
public void Connect(string host, int port)
```

Parameter	Meaning
host	remote server host name or IP address.
port	TCP listening or destination port.

Return: No value is returned.

```
public async Task ConnectAsync(string host, int port, CancellationToken cancellationToken = default)
```

Parameter	Meaning
host	remote server host name or IP address.
port	TCP listening or destination port.
cancellationToken	token used to cancel the asynchronous operation.

Return: Returns `async` as declared by the API.

```
public void Send(JollyNetworkMessage message)
```

Parameter	Meaning
message	network message to send/broadcast.

Return: No value is returned.

```
public void SendText(string text)
```

Parameter	Meaning
text	text content to encode, render or send.

Return: No value is returned.

```
public void Disconnect()
```

Return: No value is returned.

```
public void Dispose()
```

Return: No value is returned.

`Networking/JollyNetworkConnection.cs`

```
public string Id { get; } = Guid.NewGuid().ToString("N");
```

Parameter	Meaning
).ToString("N"	value supplied to the operation.

Return: Returns `string` as declared by the API.

`public void Send(JollyNetworkMessage message)`

Parameter	Meaning
`message`	network message to send/broadcast.

Return: No value is returned.

`public void SendText(string text)`

Parameter	Meaning
`text`	text content to encode, render or send.

Return: No value is returned.

`public void Disconnect()`

Return: No value is returned.

`public void Dispose()`

Return: No value is returned.

`Networking/JollyNetworkMessage.cs`

`public JollyNetworkMessage(byte[] payload)`

Parameter	Meaning
`payload`	value supplied to the operation.

Return: Returns `JollyNetworkMessage` as declared by the API.

`public byte[] Payload { get; } public string Text => Encoding.UTF8.GetString(Payload);`

Parameter	Meaning
Payload	value supplied to the operation.

Return: Returns `byte[]` as declared by the API.

`public static JollyNetworkMessage FromText(string text)`

Parameter	Meaning
`text`	text content to encode, render or send.

Return: Returns `JollyNetworkMessage` as declared by the API.

`Networking/JollyNetworkServer.cs`

`public void Start(int port)`

Parameter	Meaning
`port`	TCP listening or destination port.

Return: No value is returned.

```
public void Stop()
```

Return: No value is returned.

```
public void Broadcast(JollyNetworkMessage message)
```

Parameter	Meaning
message	network message to send/broadcast.

Return: No value is returned.

```
public void BroadcastText(string text)
```

Parameter	Meaning
text	text content to encode, render or send.

Return: No value is returned.

```
public void Dispose()
```

Return: No value is returned.

`Particles/JollyParticleEmitter.cs`

```
public int Update(float deltaSeconds, List<JollyParticle> particles)
```

Parameter	Meaning
deltaSeconds	elapsed time in seconds for the current update.
particles	value supplied to the operation.

Return: Returns `int` as declared by the API.

```
public void Emit(int count, List<JollyParticle> particles)
```

Parameter	Meaning
count	number of items/frames to process.
particles	value supplied to the operation.

Return: No value is returned.

```
public void Reset()
```

Return: No value is returned.

```
public void Burst(int count, List<JollyParticle> particles)
```

Parameter	Meaning
count	number of items/frames to process.
particles	value supplied to the operation.

Return: No value is returned.

```
public void ApplyPreset(JollyParticlePreset preset)
```

Parameter	Meaning
preset	particle preset to apply.

Return: No value is returned.

`Particles/JollyParticleRenderer.cs`

```
public JollyParticleRenderer(JollyParticleSystem particleSystem, Func<JollyTexture, JollySprite>
spriteFactory, Func<JollyColor, JollyTexture> textureFactory)
```

Parameter	Meaning
`particleSystem`	value supplied to the operation.
`spriteFactory`	value supplied to the operation.
`textureFactory`	value supplied to the operation.

Return: Returns `JollyParticleRenderer` as declared by the API.

```
public void Update(JollyTexture texture)
```

Parameter	Meaning
`texture`	the texture used by the sprite or render operation.

Return: No value is returned.

```
public void Clear()
```

Return: No value is returned.

`Particles/JollyParticleSystem.cs`

```
public void AddEmitter(JollyParticleEmitter emitter)
```

Parameter	Meaning
`emitter`	value supplied to the operation.

Return: No value is returned.

```
public void RemoveEmitter(JollyParticleEmitter emitter)
```

Parameter	Meaning
`emitter`	value supplied to the operation.

Return: No value is returned.

```
public void Update(float deltaSeconds)
```

Parameter	Meaning
`deltaSeconds`	elapsed time in seconds for the current update.

Return: No value is returned.

```
public void Clear()
```

Return: No value is returned.

```
public void Emit(JollyParticleEmitter emitter, int count)
```

Parameter	Meaning
`emitter`	value supplied to the operation.
`count`	number of items/frames to process.

Return: No value is returned.

`Physics/JollyCharacterController.cs`

```
public JollyCharacterController(JollyGameObject gameObject)
```

Parameter	Meaning
`gameObject`	game object being operated on.

Return: Returns `JollyCharacterController` as declared by the API.

```
public void Reset()
```

Return: No value is returned.

```
public void Move(float direction)
```

Parameter	Meaning
`direction`	movement/emission direction.

Return: No value is returned.

```
public void Stop()
```

Return: No value is returned.

```
public void SetJumpHeld(bool held)
```

Parameter	Meaning
`held`	whether an input is currently held.

Return: No value is returned.

```
public void RequestJump()
```

Return: No value is returned.

```
public void Update(JollyGameTime gameTime)
```

Parameter	Meaning
`gameTime`	the current game-time/update information.

Return: No value is returned.

`Physics/JollyCollider.cs`

```
public JollyCollider(JollySprite sprite)
```

Parameter	Meaning
`sprite`	value supplied to the operation.

Return: Returns `JollyCollider` as declared by the API.

```
public bool CanCollideWith(JollyCollider other)
```

Parameter	Meaning
`other`	the other game object or collider involved in the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool CollidesWith(JollyCollider other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool Check(JollyCollider other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

`Physics/JollyCollisionDebug.cs`

```
public static bool DrawColliders { get; set; } public static void CollisionEnter(JollyGameObject objectA, JollyGameObject objectB)
```

Parameter	Meaning
objectA	value supplied to the operation.
objectB	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public static void CollisionStay(JollyGameObject objectA, JollyGameObject objectB)
```

Parameter	Meaning
objectA	value supplied to the operation.
objectB	value supplied to the operation.

Return: No value is returned.

```
public static void CollisionExit(JollyGameObject objectA, JollyGameObject objectB)
```

Parameter	Meaning
objectA	value supplied to the operation.
objectB	value supplied to the operation.

Return: No value is returned.

```
public static void TriggerEnter(JollyGameObject objectA, JollyGameObject objectB)
```

Parameter	Meaning
objectA	value supplied to the operation.
objectB	value supplied to the operation.

Return: No value is returned.

```
public static void TriggerStay(JollyGameObject objectA, JollyGameObject objectB)
```

Parameter	Meaning
objectA	value supplied to the operation.
objectB	value supplied to the operation.

Return: No value is returned.

```
public static void TriggerExit(JollyGameObject objectA, JollyGameObject objectB)
```

Parameter	Meaning
objectA	value supplied to the operation.
objectB	value supplied to the operation.

Return: No value is returned.

```
public static void PixelHit(JollySprite spriteA, JollySprite spriteB, int x, int y)
```

Parameter	Meaning
spriteA	value supplied to the operation.
spriteB	value supplied to the operation.
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: No value is returned.

Physics/JollyCollisionResult.cs

```
public JollySprite SpriteA { get; } public JollySprite SpriteB { get; } public bool Collided { get; }
public JollyCollisionResult(JollySprite spriteA, JollySprite spriteB, bool collided)
```

Parameter	Meaning
spriteA	value supplied to the operation.
spriteB	value supplied to the operation.
collided	value supplied to the operation.

Return: Returns `JollySprite` as declared by the API.

Physics/JollyCollisionSystem.cs

```
public static bool Check(JollyCollider colliderA, JollyCollider colliderB)
```

Parameter	Meaning
colliderA	value supplied to the operation.
colliderB	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

Physics/JollyPhysicsSystem.cs

```
public static void Update(JollyGameObject gameObject, JollyGameTime gameTime)
```

Parameter	Meaning
gameObject	game object being operated on.
gameTime	the current game-time/update information.

Return: No value is returned.

```
public static void ResolveTileMap(JollyGameObject gameObject, JollyTileMap tileMap, float tileMapX, float
tileMapY, float deltaTime)
```

Parameter	Meaning
gameObject	game object being operated on.
tileMap	value supplied to the operation.
tileMapX	value supplied to the operation.
tileMapY	value supplied to the operation.
deltaTime	elapsed time in seconds for the current update.

Return: No value is returned.

```
public static bool Jump(JollyGameObject gameObject, float velocity)
```

Parameter	Meaning
gameObject	game object being operated on.
velocity	velocity vector or initial velocity.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public static void Resolve(JollyGameObject objectA, JollyGameObject objectB, float deltaTime)
```

Parameter	Meaning
objectA	value supplied to the operation.
objectB	value supplied to the operation.
deltaTime	elapsed time in seconds for the current update.

Return: No value is returned.

```
Physics/JollyProjectileObject.cs
```

```
public bool IsExpired { get; private set; } public JollyProjectileObject(JollySprite sprite) :
base(sprite)
```

Parameter	Meaning
base(sprite	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public override void Initialize()
```

Return: Returns `override` as declared by the API.

```
public override void Update(JollyGameTime gameTime)
```

Parameter	Meaning
gameTime	the current game-time/update information.

Return: Returns `override` as declared by the API.

```
public override void OnCollisionEnter(JollyGameObject other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `override` as declared by the API.

`Rendering/Camera/Camera2D.cs`

```
public void SetViewport(float width, float height)
```

Parameter	Meaning
`width`	width in engine/world pixels or source pixels, depending on the API.
`height`	height in engine/world pixels or source pixels, depending on the API.

Return: No value is returned.

```
public void SetBounds(float left, float top, float right, float bottom)
```

Parameter	Meaning
`left`	left world bound.
`top`	top world bound.
`right`	right world bound.
`bottom`	bottom world bound.

Return: No value is returned.

```
public void ClearBounds()
```

Return: No value is returned.

```
public void Follow(Vector2 targetPosition)
```

Parameter	Meaning
`targetPosition`	target world position for camera following.

Return: No value is returned.

```
public void FollowSmooth(Vector2 targetPosition, float deltaSeconds)
```

Parameter	Meaning
`targetPosition`	target world position for camera following.
`deltaSeconds`	elapsed time in seconds for the current update.

Return: No value is returned.

```
public Vector2 WorldToScreen(Vector2 worldPosition)
```

Parameter	Meaning
`worldPosition`	world-space position.

Return: Returns `Vector2` as declared by the API.

```
public Vector2 ScreenToWorld(Vector2 screenPosition)
```

Parameter	Meaning
`screenPosition`	screen-space position.

Return: Returns `Vector2` as declared by the API.

```
public Matrix3x2 GetWorldToScreenMatrix()
```

Return: Returns `Matrix3x2` as declared by the API.

```
public CameraState CaptureState()
```

Return: Returns `CameraState` as declared by the API.

```
public void Shake(float intensity, float duration)
```

Parameter	Meaning
`intensity`	strength of a visual/physical effect.
`duration`	duration in seconds.

Return: No value is returned.

```
public void UpdateShake(float deltaSeconds)
```

Parameter	Meaning
`deltaSeconds`	elapsed time in seconds for the current update.

Return: No value is returned.

`Rendering/Camera/CameraState.cs`

```
public Vector2 Position { get; } public float Zoom { get; } public float Rotation { get; } public Vector2 ViewportSize { get; } public CameraState( Vector2 position, float zoom, float rotation, Vector2 viewportSize)
```

Parameter	Meaning
`position`	world or screen position.
`zoom`	value supplied to the operation.
`rotation`	rotation in radians unless the containing API specifies otherwise.
`viewportSize`	render viewport dimensions.

Return: Returns `Vector2` as declared by the API.

```
public Matrix3x2 GetWorldToScreenMatrix()
```

Return: Returns `Matrix3x2` as declared by the API.

```
public Vector2 WorldToScreen(Vector2 worldPosition)
```

Parameter	Meaning
`worldPosition`	world-space position.

Return: Returns `Vector2` as declared by the API.

```
public Vector2 ScreenToWorld(Vector2 screenPosition)
```

Parameter	Meaning
`screenPosition`	screen-space position.

Return: Returns `Vector2` as declared by the API.

Rendering/DirectX/DirectXControl.xaml.cs

public JollyDiagnostics? Diagnostics { get; set; } public DirectXControl()

Return: Returns `JollyDiagnostics?` as declared by the API.

public static readonly System.Windows.DependencyProperty FPSProperty = System.Windows.DependencyProperty.Register(nameof(FPS), typeof(double), typeof(DirectXControl), new System.Windows.PropertyMetadata(0.0));

Parameter	Meaning
nameof(FPS)	value supplied to the operation.
typeof(double)	value supplied to the operation.
typeof(DirectXControl)	value supplied to the operation.
System.Windows.PropertyMetadata(0.0)	value supplied to the operation.

Return: Returns `readonly` as declared by the API.

public static readonly System.Windows.DependencyProperty FrameTimeProperty = System.Windows.DependencyProperty.Register(nameof(FrameTime), typeof(double), typeof(DirectXControl), new System.Windows.PropertyMetadata(0.0));

Parameter	Meaning
nameof(FrameTime)	value supplied to the operation.
typeof(double)	value supplied to the operation.
typeof(DirectXControl)	value supplied to the operation.
System.Windows.PropertyMetadata(0.0)	value supplied to the operation.

Return: Returns `readonly` as declared by the API.

public async Task WaitForReadyAsync()

Return: Returns `async` as declared by the API.

public void LoadImage(string filePath)

Parameter	Meaning
filePath	path to the source file.

Return: No value is returned.

public void SetImage(BitmapSource source)

Parameter	Meaning
source	source image/stream/resource.

Return: No value is returned.

public void SetImage(byte[] pixels, int width, int height)

Parameter	Meaning
pixels	pixel buffer.
width	width in engine/world pixels or source pixels, depending on the API.

Parameter	Meaning
`height`	height in engine/world pixels or source pixels, depending on the API.

Return: No value is returned.

```
public void SetImage(DirectXImage image)
```

Parameter	Meaning
`image`	value supplied to the operation.

Return: No value is returned.

```
public void ClearScreen()
```

Return: No value is returned.

```
public void Clear()
```

Return: No value is returned.

```
public DirectXTexture CreateTexture(string filePath)
```

Parameter	Meaning
`filePath`	path to the source file.

Return: Returns `DirectXTexture` as declared by the API.

```
public DirectXTexture CreateTexture(DirectXImage image)
```

Parameter	Meaning
`image`	value supplied to the operation.

Return: Returns `DirectXTexture` as declared by the API.

```
public DirectXJollyRenderer CreateJollyRenderer()
```

Return: Returns `DirectXJollyRenderer` as declared by the API.

`Rendering/DirectX/DirectXImage.cs`

```
public int Width { get; } public int Height { get; } public ReadOnlyMemory<byte> Pixels { get; } public DirectXImage(int width, int height, ReadOnlyMemory<byte> pixels)
```

Parameter	Meaning
`width`	width in engine/world pixels or source pixels, depending on the API.
`height`	height in engine/world pixels or source pixels, depending on the API.
`pixels`	pixel buffer.

Return: Returns `int` as declared by the API.

```
public static DirectXImage FromFile(string filePath)
```

Parameter	Meaning
`filePath`	path to the source file.

Return: Returns `DirectXImage` as declared by the API.

```
public static DirectXImage FromBitmapSource(BitmapSource source)
```

Parameter	Meaning
source	source image/stream/resource.

Return: Returns `DirectXImage` as declared by the API.

`Rendering/DirectX/DirectXRenderer.cs`

```
public SpriteVertex(float x, float y, float z, float u, float v)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.
z	value supplied to the operation.
u	value supplied to the operation.
v	value supplied to the operation.

Return: Returns `SpriteVertex` as declared by the API.

```
public LineVertex(float x, float y, JollyColor color, float edge, float coreRatio)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.
color	colour applied to the resource.
edge	value supplied to the operation.
coreRatio	value supplied to the operation.

Return: Returns `LineVertex` as declared by the API.

```
public double FPS => Volatile.Read(ref _fps);
```

Parameter	Meaning
_fps	value supplied to the operation.

Return: Returns `double` as declared by the API.

```
public double FrameTime => Volatile.Read(ref _frameTime);
```

Parameter	Meaning
_frameTime	value supplied to the operation.

Return: Returns `double` as declared by the API.

```
public void Initialize(IntPtr hwnd, int width, int height)
```

Parameter	Meaning
hwnd	value supplied to the operation.
width	width in engine/world pixels or source pixels, depending on the API.
height	height in engine/world pixels or source pixels, depending on the API.

Return: No value is returned.

```
public void StartRenderLoop()
```

Return: No value is returned.

```
public void StopRenderLoop()
```

Return: No value is returned.

```
public void SetStretchMode(DirectXStretchMode mode)
```

Parameter	Meaning
mode	operation or collision mode to use.

Return: No value is returned.

```
public void SetImageFilter(DirectXImageFilter filter)
```

Parameter	Meaning
filter	texture sampling filter.

Return: No value is returned.

```
public void LoadImage(string filePath)
```

Parameter	Meaning
filePath	path to the source file.

Return: No value is returned.

```
public void SetImage(BitmapSource source)
```

Parameter	Meaning
source	source image/stream/resource.

Return: No value is returned.

```
public void SetImage(byte[] pixels, int width, int height)
```

Parameter	Meaning
pixels	pixel buffer.
width	width in engine/world pixels or source pixels, depending on the API.
height	height in engine/world pixels or source pixels, depending on the API.

Return: No value is returned.

```
public void SetImage(DirectXImage image)
```

Parameter	Meaning
image	value supplied to the operation.

Return: No value is returned.

```
public DirectXTexture CreateTextureResource(string filePath)
```

Parameter	Meaning
`filePath`	path to the source file.

Return: Returns `DirectXTexture` as declared by the API.

```
public DirectXTexture CreateTextureResource(DirectXImage image)
```

Parameter	Meaning
`image`	value supplied to the operation.

Return: Returns `DirectXTexture` as declared by the API.

```
public void SubmitRenderFrame(JollyRenderFrame frame, Camera2D camera)
```

Parameter	Meaning
`frame`	value supplied to the operation.
`camera`	camera used for coordinate conversion.

Return: No value is returned.

```
public void Clear()
```

Return: No value is returned.

```
public void ClearScreen()
```

Return: No value is returned.

```
public void Render()
```

Return: No value is returned.

```
public void Resize(int width, int height)
```

Parameter	Meaning
`width`	width in engine/world pixels or source pixels, depending on the API.
`height`	height in engine/world pixels or source pixels, depending on the API.

Return: No value is returned.

```
public ImageVertex(float x, float y, float z, float u, float v)
```

Parameter	Meaning
`x`	X coordinate or horizontal position.
`y`	Y coordinate or vertical position.
`z`	value supplied to the operation.
`u`	value supplied to the operation.
`v`	value supplied to the operation.

Return: Returns `ImageVertex` as declared by the API.

```
public void Dispose()
```

Return: No value is returned.

`Rendering/DirectX/DirectXSpriteBinding.cs`

```
public JollySprite Sprite { get; } public DirectXSpriteState State { get; } public
DirectXSpriteBinding(JollySprite sprite, DirectXTexture texture)
```

Parameter	Meaning
`sprite`	value supplied to the operation.
`texture`	the texture used by the sprite or render operation.

Return: Returns `JollySprite` as declared by the API.

```
public void Synchronize()
```

Return: No value is returned.

`Rendering/DirectX/DirectXSpriteState.cs`

```
public DirectXSpriteState(DirectXTexture texture)
```

Parameter	Meaning
`texture`	the texture used by the sprite or render operation.

Return: Returns `DirectXSpriteState` as declared by the API.

`Rendering/DirectX/DirectXTexture.cs`

```
public DirectXImage Image { get; } internal DirectXTexture(DirectXImage image)
```

Parameter	Meaning
`image`	value supplied to the operation.

Return: Returns `DirectXImage` as declared by the API.

`Rendering/DirectX/JollyPhysicsDebugRenderer.cs`

```
public JollyPhysicsDebugRenderer(DirectXJollyRenderer renderer)
```

Parameter	Meaning
`renderer`	the renderer used to create or submit graphics resources.

Return: Returns `JollyPhysicsDebugRenderer` as declared by the API.

```
public void Update(IReadOnlyList<JollyGameObject> objects)
```

Parameter	Meaning
`objects`	value supplied to the operation.

Return: No value is returned.

```
public void Update(IReadOnlyList<JollyGameObject> objects, JollyTileMap? tileMap, float tileMapX, float
tileMapY)
```

Parameter	Meaning
`objects`	value supplied to the operation.
`tileMap`	value supplied to the operation.
`tileMapX`	value supplied to the operation.

Parameter	Meaning
`tileMapY`	value supplied to the operation.

Return: No value is returned.

`Rendering/DirectXJollyRenderer.cs`

```
public Camera2D Camera { get; } = new();
```

Return: Returns `Camera2D` as declared by the API.

```
public DirectXJollyRenderer(DirectXControl control)
```

Parameter	Meaning
`control`	the WPF DirectX control hosting the renderer.

Return: Returns `DirectXJollyRenderer` as declared by the API.

```
public override JollyTexture LoadTexture(string filePath)
```

Parameter	Meaning
`filePath`	path to the source file.

Return: Returns `override` as declared by the API.

```
public JollyTexture CreateSolidTexture(byte red, byte green, byte blue, byte alpha = 255)
```

Parameter	Meaning
`red`	red colour component (0..255).
`green`	green colour component (0..255).
`blue`	blue colour component (0..255).
`alpha`	alpha component, normally 0..255 for byte-based creation APIs.

Return: Returns `JollyTexture` as declared by the API.

```
public JollyTexture CreateCircleTexture(byte red, byte green, byte blue, byte alpha = 255, int size = 32)
```

Parameter	Meaning
`red`	red colour component (0..255).
`green`	green colour component (0..255).
`blue`	blue colour component (0..255).
`alpha`	alpha component, normally 0..255 for byte-based creation APIs.
`size`	size of the generated or configured object.

Return: Returns `JollyTexture` as declared by the API.

```
public override JollySprite CreateSprite(JollyTexture texture)
```

Parameter	Meaning
`texture`	the texture used by the sprite or render operation.

Return: Returns `override` as declared by the API.

```
public void SubmitFrame(JollyRenderFrame frame)
```

Parameter	Meaning
`frame`	value supplied to the operation.

Return: No value is returned.

```
public JollyTexture CreateTextTexture(string text, string fontFamily, float fontSize, JollyColor color,
bool bold = false, bool italic = false)
```

Parameter	Meaning
`text`	text content to encode, render or send.
`fontFamily`	font family name used to render text.
`fontSize`	font size used to render text.
`color`	colour applied to the resource.
`bold`	whether text should use a bold typeface.
`italic`	whether text should use an italic typeface.

Return: Returns `JollyTexture` as declared by the API.

```
public void Resize(float width, float height)
```

Parameter	Meaning
`width`	width in engine/world pixels or source pixels, depending on the API.
`height`	height in engine/world pixels or source pixels, depending on the API.

Return: No value is returned.

`Rendering/JollyRenderSubmission.cs`

```
public JollyRenderFrame Frame { get; } public CameraState Camera { get; } public
JollyRenderSubmission(JollyRenderFrame frame, Camera2D camera)
```

Parameter	Meaning
`frame`	value supplied to the operation.
`camera`	camera used for coordinate conversion.

Return: Returns `JollyRenderFrame` as declared by the API.

`Rendering/JollySpriteSheet.cs`

```
public int Index { get; } public int Column { get; } public int Row { get; } public int X { get; } public
int Y { get; } public int Width { get; } public int Height { get; } internal JollySpriteSheetFrame(int
index, int column, int row, int x, int y, int width, int height)
```

Parameter	Meaning
`index`	zero-based index.
`column`	value supplied to the operation.
`row`	value supplied to the operation.
`x`	X coordinate or horizontal position.

Parameter	Meaning
y	Y coordinate or vertical position.
width	width in engine/world pixels or source pixels, depending on the API.
height	height in engine/world pixels or source pixels, depending on the API.

Return: Returns `int` as declared by the API.

```
public JollySpriteSheetOrder Order { get; } public JollySpriteSheet(JollyTexture texture, int frameWidth, int frameHeight, JollySpriteSheetOrder order = JollySpriteSheetOrder.Horizontal)
```

Parameter	Meaning
texture	the texture used by the sprite or render operation.
frameWidth	value supplied to the operation.
frameHeight	value supplied to the operation.
order	value supplied to the operation.

Return: Returns `JollySpriteSheetOrder` as declared by the API.

```
public JollySpriteSheetFrame GetFrame(int index)
```

Parameter	Meaning
index	zero-based index.

Return: Returns `JollySpriteSheetFrame` as declared by the API.

```
Rendering/JollyTextureManager.cs
```

```
public JollyTextureManager(JollyRenderer renderer)
```

Parameter	Meaning
renderer	the renderer used to create or submit graphics resources.

Return: Returns `JollyTextureManager` as declared by the API.

```
public JollyTexture Load(string filePath)
```

Parameter	Meaning
filePath	path to the source file.

Return: Returns `JollyTexture` as declared by the API.

```
public bool TryGet(string filePath, out JollyTexture? texture)
```

Parameter	Meaning
filePath	path to the source file.
texture	the texture used by the sprite or render operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool Contains(string filePath)
```

Parameter	Meaning
filePath	path to the source file.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void Clear()
```

Return: No value is returned.

```
Save/JollyGameObjectSaveState.cs
```

```
public bool Active { get; set; } public JollyVector2 Position { get; set; } public JollyVector2 Scale { get; set; } public float Rotation { get; set; } public bool Visible { get; set; } public JollyVector2 Velocity { get; set; } public JollyVector2 Acceleration { get; set; } public JollyVector2 Gravity { get; set; } public float Drag { get; set; } public bool UsePhysics { get; set; } public bool UseGravity { get; set; } public bool IsGrounded { get; set; } public bool WasGrounded { get; set; } public float GroundFriction { get; set; } public float AirFriction { get; set; } public float Restitution { get; set; } public JollyColliderSaveState? Collider { get; set; } public static JollyGameObjectSaveState Capture(JollyGameObject gameObject)
```

Parameter	Meaning
gameObject	game object being operated on.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void ApplyTo(JollyGameObject gameObject)
```

Parameter	Meaning
gameObject	game object being operated on.

Return: No value is returned.

```
public bool Enabled { get; set; } public bool IsTrigger { get; set; } public JollyCollisionLayer Layer { get; set; } public JollyCollisionLayer Mask { get; set; } public JollyColliderShape Shape { get; set; } public float Width { get; set; } public float Height { get; set; } public float Radius { get; set; } public byte AlphaThreshold { get; set; } public static JollyColliderSaveState Capture(JollyCollider collider)
```

Parameter	Meaning
collider	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void ApplyTo(JollyCollider collider)
```

Parameter	Meaning
collider	value supplied to the operation.

Return: No value is returned.

```
Save/JollySaveManager.cs
```

```
public string SaveDirectory { get; } public JollySaveManager(string? saveDirectory = null)
```

Parameter	Meaning
saveDirectory	value supplied to the operation.

Return: Returns `string` as declared by the API.

```
public void SaveGame<T>(string slot, T state, int gameVersion = 1)
```

Parameter	Meaning
`slot`	save-slot identifier.
`state`	state object to serialize or apply.
`gameVersion`	game-defined save version.

Return: No value is returned.

```
public T LoadGame<T>(string slot, int? expectedGameVersion = null)
```

Parameter	Meaning
`slot`	save-slot identifier.
`expectedGameVersion`	optional game version that must match when loading.

Return: Returns `T` as declared by the API.

```
public bool TryLoadGame<T>(string slot, out T? state, int? expectedGameVersion = null)
```

Parameter	Meaning
`slot`	save-slot identifier.
`state`	state object to serialize or apply.
`expectedGameVersion`	optional game version that must match when loading.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool Exists(string slot)
```

Parameter	Meaning
`slot`	save-slot identifier.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool HasBackup(string slot)
```

Parameter	Meaning
`slot`	save-slot identifier.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool RestoreBackup(string slot)
```

Parameter	Meaning
`slot`	save-slot identifier.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void Delete(string slot)
```

Parameter	Meaning
`slot`	save-slot identifier.

Return: No value is returned.

```
public IReadOnlyList<string> GetSlots()
```

Return: Returns `IReadOnlyList<string>` as declared by the API.

```
public JollySaveException(string message) : base(message)
```

Parameter	Meaning
base(message	value supplied to the operation.

Return: Returns `JollySaveException` as declared by the API.

```
public JollySaveException(string message, Exception innerException) : base(message, innerException)
```

Parameter	Meaning
message	network message to send/broadcast.
innerException	value supplied to the operation.

Return: Returns `JollySaveException` as declared by the API.

`Save/JollySceneSaveState.cs`

```
public List<JollyGameObjectSaveState> Objects { get; set; } = new();
```

Return: Returns `List<JollyGameObjectSaveState>` as declared by the API.

```
public static JollySceneSaveState Capture(JollyScene scene)
```

Parameter	Meaning
scene	value supplied to the operation.

Return: Returns `JollySceneSaveState` as declared by the API.

```
public int ApplyTo(JollyScene scene)
```

Parameter	Meaning
scene	value supplied to the operation.

Return: Returns `int` as declared by the API.

`Scene/JollyScene.cs`

```
public void SetTileMap(JollyTileMap tileMap, float x, float y)
```

Parameter	Meaning
tileMap	value supplied to the operation.
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: No value is returned.

```
public void ClearTileMap()
```

Return: No value is returned.

```
public virtual void OnEnter()
```

Return: Returns `virtual` as declared by the API.

```
public virtual void OnExit()
```

Return: Returns `virtual` as declared by the API.

```
public virtual void Initialize()
```

Return: Returns `virtual` as declared by the API.

```
public void Add(JollyGameObject gameObject)
```

Parameter	Meaning
`gameObject`	game object being operated on.

Return: No value is returned.

```
public void Spawn(JollyGameObject gameObject)
```

Parameter	Meaning
`gameObject`	game object being operated on.

Return: No value is returned.

```
public void Remove(JollyGameObject gameObject)
```

Parameter	Meaning
`gameObject`	game object being operated on.

Return: No value is returned.

```
public void Despawn(JollyGameObject gameObject)
```

Parameter	Meaning
`gameObject`	game object being operated on.

Return: No value is returned.

```
public virtual void Update(JollyGameTime gameTime)
```

Parameter	Meaning
`gameTime`	the current game-time/update information.

Return: Returns `virtual` as declared by the API.

```
public virtual void Draw()
```

Return: Returns `virtual` as declared by the API.

```
public IReadOnlyList<JollySprite> GetSprites()
```

Return: Returns `IReadOnlyList<JollySprite>` as declared by the API.

```
public JollyGameObject? FindBySaveId(string saveId)
```

Parameter	Meaning
`saveId`	value supplied to the operation.

Return: Returns `JollyGameObject?` as declared by the API.

```
public JollyGameObject? Find(string name)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `JollyGameObject?` as declared by the API.

```
public T? Find<T>(string name) where T : JollyGameObject
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `T?` as declared by the API.

```
public IReadOnlyList<T> FindAll<T>() where T : JollyGameObject
```

Return: Returns `IReadOnlyList<T>` as declared by the API.

```
public bool Contains(JollyGameObject gameObject)
```

Parameter	Meaning
gameObject	game object being operated on.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public virtual void Shutdown()
```

Return: Returns `virtual` as declared by the API.

```
public static void ResolveTileMap(JollyGameObject gameObject, JollyTileMap tileMap, float tileMapX, float tileMapY, float deltaTime)
```

Parameter	Meaning
gameObject	game object being operated on.
tileMap	value supplied to the operation.
tileMapX	value supplied to the operation.
tileMapY	value supplied to the operation.
deltaTime	elapsed time in seconds for the current update.

Return: No value is returned.

```
public JollyGameObject ObjectA { get; } public JollyGameObject ObjectB { get; } public CollisionPair(JollyGameObject objectA, JollyGameObject objectB)
```

Parameter	Meaning
objectA	value supplied to the operation.
objectB	value supplied to the operation.

Return: Returns `JollyGameObject` as declared by the API.

```
public bool Equals(CollisionPair other)
```

Parameter	Meaning
other	the other game object or collider involved in the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public override bool Equals(object? obj)
```

Parameter	Meaning
obj	value supplied to the operation.

Return: Returns `override` as declared by the API.

```
public override int GetHashCode()
```

Return: Returns `override` as declared by the API.

`Scene/JollySceneManager.cs`

```
public void ChangeScene(JollyScene scene)
```

Parameter	Meaning
scene	value supplied to the operation.

Return: No value is returned.

```
public JollyGameObject? Find(string name)
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `JollyGameObject?` as declared by the API.

```
public T? Find<T>(string name) where T : JollyGameObject
```

Parameter	Meaning
name	name used to identify the object, asset, animation, sound or slot.

Return: Returns `T?` as declared by the API.

```
public IReadOnlyList<T> FindAll<T>() where T : JollyGameObject
```

Return: Returns `IReadOnlyList<T>` as declared by the API.

```
public void Update(JollyGameTime gameTime)
```

Parameter	Meaning
gameTime	the current game-time/update information.

Return: No value is returned.

```
public void Draw()
```

Return: No value is returned.

```
public void Shutdown()
```

Return: No value is returned.

`UI/JollyUI9Slice.cs`

```
public JollyTexture Texture { get; } public float Left { get; set; } public float Top { get; set; } public float Right { get; set; } public float Bottom { get; set; } public JollyUI9Slice(JollyTexture texture, float left, float top, float right, float bottom)
```

Parameter	Meaning
texture	the texture used by the sprite or render operation.
left	left world bound.
top	top world bound.
right	right world bound.
bottom	bottom world bound.

Return: Returns `JollyTexture` as declared by the API.

`UI/JollyUIAnimation.cs`

```
public JollyUIElement Element { get; } public JollyUIAnimationProperty Property { get; } public float Duration { get; } public bool IsComplete { get; private set; } internal JollyUIAnimation(JollyUIElement element, JollyUIAnimationProperty property, Vector2 start, Vector2 end, float duration)
```

Parameter	Meaning
element	value supplied to the operation.
property	UI animation property.
start	line start position.
end	line end position.
duration	duration in seconds.

Return: Returns `JollyUIElement` as declared by the API.

`UI/JollyUIButton.cs`

```
public JollyColor NormalColor { get; set; } = JollyColor.FromBytes(60, 60, 60, 255);
```

Parameter	Meaning
60	value supplied to the operation.
60	value supplied to the operation.
60	value supplied to the operation.
255	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public JollyColor HoverColor { get; set; } = JollyColor.FromBytes(80, 80, 80, 255);
```

Parameter	Meaning
80	value supplied to the operation.
80	value supplied to the operation.
80	value supplied to the operation.
255	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public JollyColor PressedColor { get; set; } = JollyColor.FromBytes(40, 40, 40, 255);
```

Parameter	Meaning
40	value supplied to the operation.

Parameter	Meaning
40	value supplied to the operation.
40	value supplied to the operation.
255	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public JollyColor DisabledColor { get; set; } = JollyColor.FromBytes(35, 35, 35, 255);
```

Parameter	Meaning
35	value supplied to the operation.
35	value supplied to the operation.
35	value supplied to the operation.
255	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

`UI/JollyUIElement.cs`

```
public bool ContainsPoint(Vector2 point)
```

Parameter	Meaning
point	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void AddChild(JollyUIElement child)
```

Parameter	Meaning
child	value supplied to the operation.

Return: No value is returned.

```
public bool RemoveChild(JollyUIElement child)
```

Parameter	Meaning
child	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void RemoveAllChildren()
```

Return: No value is returned.

```
public void SetMargin(float left, float top, float right, float bottom)
```

Parameter	Meaning
left	left world bound.
top	top world bound.
right	right world bound.
bottom	bottom world bound.

Return: No value is returned.

`UI/JollyUIManager.cs`

`public float ScreenHeight { get; private set; } public void SetScreenSize(float width, float height)`

Parameter	Meaning
`width`	width in engine/world pixels or source pixels, depending on the API.
`height`	height in engine/world pixels or source pixels, depending on the API.

Return: Returns `float` as declared by the API.

`public void Add(JollyUIElement element)`

Parameter	Meaning
`element`	value supplied to the operation.

Return: No value is returned.

`public bool Remove(JollyUIElement element)`

Parameter	Meaning
`element`	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

`public void Clear()`

Return: No value is returned.

`public JollyUIElement? HitTest(Vector2 point)`

Parameter	Meaning
`point`	value supplied to the operation.

Return: Returns `JollyUIElement?` as declared by the API.

`public void MouseMove(float x, float y)`

Parameter	Meaning
`x`	X coordinate or horizontal position.
`y`	Y coordinate or vertical position.

Return: No value is returned.

`public void MouseDown()`

Return: No value is returned.

`public void MouseUp()`

Return: No value is returned.

`public JollyUIAnimation AnimatePosition(JollyUIElement element, Vector2 target, float duration)`

Parameter	Meaning
`element`	value supplied to the operation.
`target`	value supplied to the operation.

Parameter	Meaning
duration	duration in seconds.

Return: Returns `JollyUIAnimation` as declared by the API.

```
public JollyUIAnimation AnimateSize(JollyUIElement element, Vector2 target, float duration)
```

Parameter	Meaning
element	value supplied to the operation.
target	value supplied to the operation.
duration	duration in seconds.

Return: Returns `JollyUIAnimation` as declared by the API.

```
public JollyUIAnimation AnimateOpacity(JollyUIElement element, float target, float duration)
```

Parameter	Meaning
element	value supplied to the operation.
target	value supplied to the operation.
duration	duration in seconds.

Return: Returns `JollyUIAnimation` as declared by the API.

```
public void Update(float deltaSeconds)
```

Parameter	Meaning
deltaSeconds	elapsed time in seconds for the current update.

Return: No value is returned.

```
public void UpdateLayout()
```

Return: No value is returned.

`UI/JollyUIRenderer.cs`

```
public JollyUIRenderer(JollyUIManager uiManager, Func<JollyTexture, JollySprite> spriteFactory,
Func<byte, byte, byte, byte, JollyTexture> textureFactory, Func<string, string, float, JollyColor, bool,
bool, JollyTexture> textTextureFactory)
```

Parameter	Meaning
uiManager	value supplied to the operation.
spriteFactory	value supplied to the operation.
textureFactory	value supplied to the operation.
textTextureFactory	value supplied to the operation.

Return: Returns `JollyUIRenderer` as declared by the API.

```
public void Update()
```

Return: No value is returned.

```
public void Clear()
```

Return: No value is returned.

`UI/JollyUISlider.cs`

```
public JollyColor TrackColor { get; set; } = JollyColor.FromBytes(50, 50, 50, 255);
```

Parameter	Meaning
50	value supplied to the operation.
50	value supplied to the operation.
50	value supplied to the operation.
255	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

```
public JollyColor FillColor { get; set; } = JollyColor.FromBytes(80, 160, 240, 255);
```

Parameter	Meaning
80	value supplied to the operation.
160	value supplied to the operation.
240	value supplied to the operation.
255	value supplied to the operation.

Return: Returns `JollyColor` as declared by the API.

`World/JollyTileMap.cs`

```
public int Width { get; } public int Height { get; } public float TileWidth { get; } public float
TileHeight { get; } public JollyTileMap(int width, int height, float tileWidth, float tileHeight)
```

Parameter	Meaning
width	width in engine/world pixels or source pixels, depending on the API.
height	height in engine/world pixels or source pixels, depending on the API.
tileWidth	value supplied to the operation.
tileHeight	value supplied to the operation.

Return: Returns `int` as declared by the API.

```
public void NotifyHazardContact(JollyGameObject gameObject, int tileX, int tileY, int tileIndex)
```

Parameter	Meaning
gameObject	game object being operated on.
tileX	value supplied to the operation.
tileY	value supplied to the operation.
tileIndex	value supplied to the operation.

Return: No value is returned.

```
public bool IsSolid(int x, int y)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void SetSolid(int x, int y, bool solid)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.
solid	value supplied to the operation.

Return: No value is returned.

```
public int GetTile(int x, int y)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: Returns `int` as declared by the API.

```
public void SetTile(int x, int y, int tileIndex)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.
tileIndex	value supplied to the operation.

Return: No value is returned.

```
public Vector2 GetTilePosition(int x, int y)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: Returns `Vector2` as declared by the API.

```
public Vector2 WorldToTile(float x, float y)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: Returns `Vector2` as declared by the API.

```
public bool IsSolidAtWorldPosition(float x, float y)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public bool TryGetTileAtWorldPosition(float x, float y, out int tileIndex)
```

Parameter	Meaning
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.
tileIndex	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void SetTexture(JollyTexture texture)
```

Parameter	Meaning
texture	the texture used by the sprite or render operation.

Return: No value is returned.

```
public bool TryGetTileSourceRectangle(int tileIndex, out int x, out int y, out int width, out int height)
```

Parameter	Meaning
tileIndex	value supplied to the operation.
x	X coordinate or horizontal position.
y	Y coordinate or vertical position.
width	width in engine/world pixels or source pixels, depending on the API.
height	height in engine/world pixels or source pixels, depending on the API.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void SetAnimatedTile(int tileIndex, int startFrame, int frameCount, float framesPerSecond, bool loop = true)
```

Parameter	Meaning
tileIndex	value supplied to the operation.
startFrame	first sprite-sheet frame used by the animation.
frameCount	number of frames used by the animation.
framesPerSecond	value supplied to the operation.
loop	whether playback loops.

Return: No value is returned.

```
public void RemoveAnimatedTile(int tileIndex)
```

Parameter	Meaning
tileIndex	value supplied to the operation.

Return: No value is returned.

```
public bool IsAnimatedTile(int tileIndex)
```

Parameter	Meaning
tileIndex	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public void SetConveyorTile(int tileIndex, float speed)
```

Parameter	Meaning
tileIndex	value supplied to the operation.
speed	value supplied to the operation.

Return: No value is returned.

```
public void RemoveConveyorTile(int tileIndex)
```

Parameter	Meaning
tileIndex	value supplied to the operation.

Return: No value is returned.

```
public bool IsConveyorTile(int tileIndex)
```

Parameter	Meaning
tileIndex	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public float GetConveyorSpeed(int tileIndex)
```

Parameter	Meaning
tileIndex	value supplied to the operation.

Return: Returns `float` as declared by the API.

```
public int GetAnimatedTileIndex(int tileIndex, float elapsedSeconds)
```

Parameter	Meaning
tileIndex	value supplied to the operation.
elapsedSeconds	value supplied to the operation.

Return: Returns `int` as declared by the API.

```
public void SetHazardTile(int tileIndex, bool hazard)
```

Parameter	Meaning
tileIndex	value supplied to the operation.
hazard	value supplied to the operation.

Return: No value is returned.

```
public bool IsHazardTile(int tileIndex)
```

Parameter	Meaning
tileIndex	value supplied to the operation.

Return: Returns `true` when the requested condition/operation succeeds; otherwise `false`.

```
public int StartFrame { get; } public int FrameCount { get; } public float FramesPerSecond { get; } public bool Loop { get; } public JollyTileAnimation(int startFrame, int frameCount, float framesPerSecond, bool loop)
```

Parameter	Meaning
`startFrame`	first sprite-sheet frame used by the animation.
`frameCount`	number of frames used by the animation.
`framesPerSecond`	value supplied to the operation.
`loop`	whether playback loops.

Return: Returns `int` as declared by the API.

`World/JollyTileMapRenderer.cs`

```
public int Layer { get; set; } public float X { get; set; } public float Y { get; set; } public JollyTileMapRenderer(JollyTileMap tileMap, Func<JollyTexture, JollySprite> spriteFactory)
```

Parameter	Meaning
`tileMap`	value supplied to the operation.
`spriteFactory`	value supplied to the operation.

Return: Returns `int` as declared by the API.

```
public void Rebuild()
```

Return: No value is returned.

```
public void Update(Camera2D camera, float deltaSeconds)
```

Parameter	Meaning
`camera`	camera used for coordinate conversion.
`deltaSeconds`	elapsed time in seconds for the current update.

Return: No value is returned.